智能软件开发的组织与交互框架：面向能力放大与持续演化

AI 员工项目*

2026 年 10 月 15 日

摘要

语言模型逐步承担软件开发中的理解、实现、协调和验证活动之后，开发系统的关键问题扩展到长期组织、经验延续以及人类如何把握工作。本文提出一个面向持续演化的智能软件开发框架：以人的目标与语义约束维持方向，以沿架构职责组织的持续 Agent 支持知识积累，以递归委派和上下文管理调整工作结构，以局部交付和自适应验证连接开发与真实使用，并通过可生成的交互界面将理解、决策和反馈放在同一工作情境中。框架将开发流程视为 AI 能力的放大器，关注模型能力提升后可承担的工程复杂度，以及长期工作中人的注意力与协调负担。文章结合 Portal 的实践动机，给出六项交互生成原则、六类典型投影指南和一个跨模块任务取消的设计示例，进一步提出多轮演化评价和资源分配方法。本文区分实践观察、设计建议与待验证假设，讨论适用条件与失败方式，为后续系统实现和经验研究提供可操作的研究议程。

关键词：智能软件开发；自主 Agent；软件演化；人机协作；生成式界面；自适应验证

*基于项目实践与人机共同讨论形成；AI 辅助研究、撰写与制图。本文为概论与设计框架，所提出的效果假设有待系统评价。

目录

1 绪论：智能软件开发的问题与愿景 5

1.1 从完成一次任务到长期发展一个系统 5
1.2 三个相互作用的目的 5
1.3 实践动机与研究范围 6
1.4 与既有研究的联系 6
1.5 文章贡献与阅读路线 7

2 目标与基本概念：开发系统作为能力放大器 8

2.1 能力放大的含义 8
2.2 核心对象与职责 8
2.3 三个相互连接的工作过程 9
2.4 人的控制成本与系统资源 9
2.5 能力边界与收益发生的条件 10
2.6 组织自主性与可理解性 10

3 需求与语义约束：在持续变化中保持软件方向 11

3.1 意图需要联系正在运行的软件 11
3.2 语义规范的内容与颗粒度 11
3.3 让目标、实现和证据可以互相定位 12
3.4 一个可使用的行为说明实例 12
3.5 规范、测试与实现发生冲突时 13
3.6 工作状态与知识的不同生命周期 13
3.7 把澄清作为持续协作的一部分 14

4 自主团队组织：架构分工、递归委派与长期经验 15

4.1 长期职责与临时任务 15
4.2 沿架构职责组织成员 15
4.3 协议是自主协作的共同依据 16
4.4 上下文分配与递归委派 16
4.5 连续身份、恢复与经验更新 17
4.6 并行工作中的协调边界 18
4.7 组织调整与独立核对 18

5 持续开发与快速交付：在并行变化中推进演化 19

5.1 持续变化是工作情境 . . . 19
5.2 确定性工具与智能判断的分工 . . . 19
5.3 验证与发布范围相互说明 . . . 19
5.4 一次交付的最小事实记录 . . . 20
5.5 环境隔离与恢复的条件性取舍 . . . 21
5.6 低成本记录与交付后的学习 . . . 21

6 自适应验证：语义规范、代码测试与真实使用 22

6.1 验证要支持什么判断 . . . 22
6.2 以完整用户任务组织 TAU . . . 22
6.3 证据在变化后的有效范围 . . . 22
6.4 何时扩大验证 . . . 23
6.5 三种不同变化下的投入选择 . . . 24
6.6 从失败到修复完成 . . . 25
6.7 让验证任务与结果可以交接 . . . 25
6.8 验证的组织与可靠性 . . . 26

7 智能交互与工作投影：让理解和控制共处 27

7.1 信息集中之后仍然存在理解成本 . . . 27
7.2 工作语义与交互投影 . . . 27
7.3 理解与控制发生在同一情境中 . . . 27
7.4 表达必须说明状态与作用 . . . 28
7.5 投影如何保持准确与连续 . . . 29
7.6 多媒体选择与生成成本 . . . 29
7.7 异步汇报与表达的持续演化 . . . 29

8 UI 与汇报生成指南：从共同原则到典型表达 31

8.1 六项共同原则 . . . 31
8.2 指南一：层次化架构与协议探索 . . . 31
8.3 指南二：围绕真实任务的功能体验 . . . 32
8.4 指南三：运行机制与可调情境 . . . 32
8.5 指南四：共同场景下的方案比较 . . . 33

8.6 指南五：阶段进展与注意力分配 . . . 33
8.7 指南六：未来演化与依赖路径 . . . 34
8.8 把指南转化为可执行的生成任务 . . . 34
8.9 指南的组合、生成与修订 . . . 35

9 综合案例：一次跨模块改进如何完成 36
9.1 案例条件与初始问题 . . . 36
9.2 形成语义约束与工作范围 . . . 36
9.3 协议设计与并发条件 . . . 37
9.4 并行实现、检查与局部交付 . . . 37
9.5 一次失败怎样改变设计 . . . 38
9.6 案例留下的可复用资产 . . . 38

10 评价与资源分配：怎样判断系统放大了能力 40
10.1 从产出计数到工程结果 . . . 40
10.2 核心评价维度与记录口径 . . . 40
10.3 多轮任务与比较条件 . . . 41
10.4 展示与交互可能成为主要投入 . . . 41
10.5 可检验的研究假设 . . . 42
10.6 一个可实施的首轮研究方案 . . . 43
10.7 哪些观察会要求修改框架 . . . 43
10.8 效度威胁与结果解释 . . . 43

11 开放问题与未来方向 45
11.1 长期方向与组织适应 . . . 45
11.2 生成交互的稳定性与个体差异 . . . 45
11.3 运行模块与开发成员的进一步结合 . . . 45
11.4 结语 . . . 46

参考文献 47

1 绪论：智能软件开发的问题与愿景

1.1 从完成一次任务到长期发展一个系统

软件开发中的一个局部任务往往容易描述：增加一个按钮、修复一个接口，或让某个测试通过。一个长期存在的软件系统却同时承载许多尚未完成的事情。已有用户依赖它的行为，新的需求持续进入，多个模块共同完成同一任务，过去的设计理由未必能从代码直接读出。开发者必须在变化中决定哪些工作值得做、怎样分配、如何确认结果，以及何时让用户实际用到成果。

当 AI 可以阅读代码、调用工具和修改实现时，这些工程判断可以逐步进入它的工作范围。由此形成的开发方式，需要同时考虑个体 Agent 的能力与整个工作系统的组织。一次成功修改证明了一个具体能力；持续接收目标、维持质量、吸收反馈并改善后续工作的能力，则需要在更长时间内观察。本文把这一持续过程作为讨论对象。

这里的“智能软件开发”指 AI 深度参与并自主承担工程判断的开发方式。所开发的软件可以是普通业务系统，也可以包含模型服务、智能助手或自动化模块。产物中是否使用 AI，与开发过程是否具有智能组织能力，是两个可以分别讨论的维度。本文重点讨论后者，并在最后讨论二者进一步结合的可能。

一个基本场景可以说明其要求。使用者希望任务平台支持可靠取消长时间运行的工作。实现涉及前端反馈、任务状态、调度、执行器和资源释放。只增加一个“取消”按钮无法完成目标；只向用户解释若干接口也不足以确认体验。开发系统需要理解取消的含义，找到责任模块，处理并发条件，验证用户看到的结果，再把这些结果以便于人判断的方式呈现。后续用户可能进一步要求保留部分输出，原有设计还要继续演化。

1.2 三个相互作用的目的

本文把智能开发系统的目的概括为三项：让人的意图更准确地进入 AI 的工作，让 AI 更自主地完成所委派的工作，以及让 AI 的成果与机制更容易被人理解和控制。三项目的相互影响。难以理解的汇报会降低反馈质量；含糊的意图会增加实现返工；执行缺乏连续性又会迫使人反复解释背景。

因此，减少人类控制成本并不等同于减少消息数量。一次短消息可能包含需要长时间调查的判断，一张复杂仪表盘也可能把大量解释工作转移给人。更合适的问题是：为了让系统继续朝期望方向发展，人需要花多少时间建立理解、比较选项、补充信息和纠正偏差？系统能否把已经表达过的偏好用于后续判断，同时在真正重要的分歧上提供有用的参与入口？

自主性也应联系结果理解。Agent 能连续调用许多工具，说明它具备执行能力；开发系统还需要判断这些动作是否服务于当前目标，是否产生可用成果，以及失败之后是否完成修复。把任务投递给另一个成员只是委派的开始。对人有意义的工作单元通常延

续到成果可体验、影响可解释、遗留问题可定位的阶段。

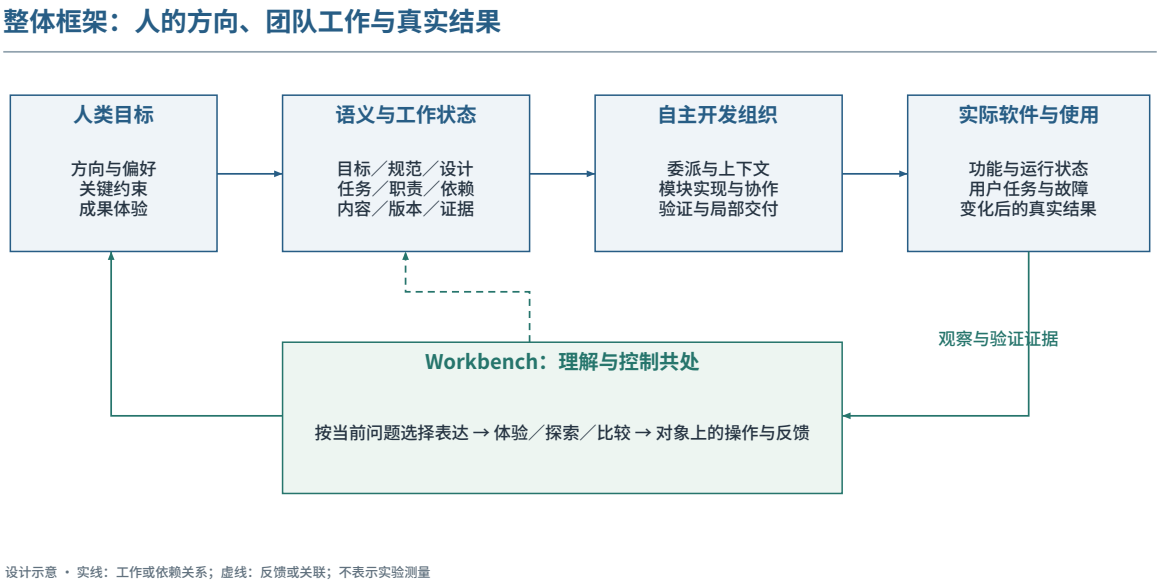


图 1 智能开发系统的整体关系。人的意图进入语义约束与自主组织；开发、验证和使用产生证据；交互投影将成果与操作带回人的工作情境。实线表示主要工作依赖，虚线表示反馈。

1.3 实践动机与研究范围

Portal 是一个围绕项目、知识、持续 Agent 和协作过程构建的系统。其开发实践提供了本文的问题来源。项目曾尝试把大量设计点、改进点和实现差异集中呈现在 WALL 界面中，使人可以监督系统的发展。随着点的数量增加，逐项查看的负担显著上升。更深的问题是，单条简短文字往往无法提供足够背景，人仍需请求解释才能判断它的含义。

另一个实践观察是，直接使用软件往往更容易产生明确反馈。操作界面同时提供目标、对象、行为和结果，人能够指出具体哪里不符合预期。此经验启发了一个设计方向：为架构、机制和未来方案也构造可探索的表达，让人能在相同情境中理解并采取行动。本文将这一方向与团队组织、持续交付和验证联系起来。

这些观察来自一个持续变化的项目及其使用者经验，不能据此推断所有团队都具有相同问题强度。本文采用概念性设计研究的写法：从明确的实践动机出发，提出机制及其作用解释，再给出可检验的评价方案。文中的过程图和界面图均为作者绘制的设计示意；第9节的完整工作过程是构造案例，用于检验框架内部的一致性。

1.4 与既有研究的联系

软件模块化研究强调将设计决定与职责组织起来，使局部变化具有可控制的影响范围 [1]。组织沟通与系统结构之间的联系也已有长期讨论 [2]。这些思想对 Agent 团队仍有意义，但持续身份、可复制上下文和低成本临时成员提供了新的组织手段。本文关注这些手段怎样服务于长期维护，以及怎样避免协调开销抵消并行收益。

语言模型 Agent 研究展示了工具交互、环境反馈和多成员协作的多种实现方向。ReAct 将推理与行动交替组织 [3]；SWE-agent 研究面向 Agent 的计算机接口如何支持软件工程任务 [4]；MetaGPT 和 AutoGen 分别探索协作工作流与多 Agent 会话组织 [5, 6]。这些工作为自主执行提供了参考。本文进一步讨论任务完成之后的持续演化、人的方向控制，以及模型升级后组织方式的适应。

在人机交互方面，逐层查看、按需获取细节与混合主动性交互已有研究基础 [7, 8]。人机 AI 交互指南也关注能力说明、纠错和随时间变化的行为 [9]。本文把这些原则应用于开发系统：由 Agent 结合当前工作生成表达，允许人在对应对象上参与，并将交互反馈持续用于更新软件和表达方式。

表 1 相关研究与本文问题的联系

研究方向	代表性研究提供的认识	本文进一步讨论的问题
模块化与组织	设计决定的分解及沟通结构的影响 [1, 2]	持续 Agent 的职责、上下文和经验怎样沿边界积累
自主编码	工具交互、仓库操作与任务结果评价 [4, 13]	多轮需求变化、交付修复与人的持续引导
多 Agent 协作	工作流和会话能够组织多成员行动 [5, 6]	何时委派、如何调整职责、怎样控制协调成本
人机交互	逐步探索、混合主动性及纠错原则 [7, 8, 9]	如何根据当前开发工作生成并维护理解与控制界面

这些研究分别提供了重要基础。本文将它们联系到同一个长期工作对象：一个需要持续维护并接受人类引导的软件系统。其提出的机制并非全部为新发明，研究价值主要在于明确组合关系、成本条件和可评价的效果。例如，较好的模块边界可以减少交付依赖，也能让架构投影更容易理解；更准确的人类反馈则可能减少下一轮实现返工。

文章也避免将传统软件工程概括为单一固定流程。成熟的人类团队同样使用持续交付、探索式测试和按职责分工。这里的变化是，更多选择这些方法并调整组织的判断可以由 AI 持续承担。需要研究的是在什么条件下这种委派有效，以及系统怎样依据实践继续改进。

1.5 文章贡献与阅读路线

本文提出一个连接组织、工程和交互的整体框架，具体贡献包括：定义面向能力放大的目标与成本视角；说明持续成员、架构职责和递归委派的关系；提出局部交付与证据驱动验证的组合方式；给出理解与控制共处的工作投影原则及典型指南；设计包含长期变化、人类负担和资源投入的评价方法。

第2节建立概念与整体关系，第3至6节讨论方向、组织、交付和验证，第7与8节讨论面向人的表达，第9节串联全过程。第10节把设计主张转化为评价问题，最后讨论开放问题。读者可以先从总体框架与案例建立直觉，再回到具体机制理解条件和取舍。

2 目标与基本概念：开发系统作为能力放大器

2.1 能力放大的含义

本文所说的能力放大，是组织方法、工具和积累使同等模型能够完成更复杂或质量更高的工程工作，并使更强模型能够进一步扩展这种范围。它包含同一能力条件下的增益，也包含能力提升后的可扩展性。判断时需要控制项目条件，观察真实产物与人的投入，不能用成员数量、执行轮数或生成代码长度替代效果。

放大的第一种来源是有效组合。不同成员深入相对独立的职责范围，共享明确的协议，在必要的地方协作。第二种来源是跨时间积累：设计理由、故障案例、工具和验证经验降低后续调查成本。第三种来源是现实反馈：交付、测试和使用让判断接触实际结果，减少重复的错误假设。第四种来源是交互质量：人更容易理解关键问题，因而能以较少努力给出有效引导。

这些来源具有条件性。并行会增加协调，记忆会过时，反馈可能不完整，生成界面也可能误导。一个有用的框架必须允许发现负收益并调整结构。某项目只需要一个足够强的 Agent 时，持续身份、证据记录和交互投影仍可使用；团队划分应服从实际工作，而非成为进入框架的先决条件。

2.2 核心对象与职责

表 2 框架中的核心对象及其持续职责

对象	含义	需要保持的联系
开发 Agent	能理解任务并通过工具采取行动的执行主体	动作与目标、结果及执行环境
持续成员	跨模型会话保有身份与职责的 Agent	维护范围、经验、未完工作
团队	围绕共同目标协作的一组成员	委派、协作关系与结果责任
软件模块	围绕相对稳定职责组织的实现与契约	内部决定与外部可依赖行为
语义规范	对目标、约束与成功含义的可理解表达	当前意图与实现、验证依据
工作证据	对特定内容和环境进行观察或检查的结果	范围、时间、适用条件与来源
交互投影	面向当前理解任务组织的视图与操作	显示对象、事实来源与操作效果

在 Portal 的术语中，持续成员称为 Resident，团队及其协作关系由 Lineage 承载，面向人的工作界面称为 Workbench。本文使用这些名称描述具体实例，同时保留一般定义。模型会话是成员使用的一种执行资源；更换会话不应自然地丢失长期职责。工作界面也只是当前任务的一种表达，同一组工作事实可以支持不同视图。

身份、知识和上下文之间需要区分。身份回答谁对某范围持续负责，知识保存经过

整理的可复用内容，上下文是某次判断能够直接访问的信息。将所有历史塞进每个会话会增加成本，也会让当前任务被无关细节淹没。组织的作用之一，是让需要的事实在适当时刻进入适当成员的上下文。

2.3 三个相互连接的工作过程

框架包含目标解释、工程执行和人机反馈三个过程。目标解释将人的表达联系到当前软件与已有约束，形成可开展的工作。工程执行组织实现、验证和交付。人机反馈选择值得呈现的变化，帮助人理解、体验和调整方向。三个过程可以并行推进，并通过稳定对象和记录协调。

例如，某成员已经在改善取消响应时，人可能反馈“真正关心的是取消后还能不能查看中间结果”。新的目标会改变输出保留策略，却不必使已经完成的状态观察工作全部失效。团队可以识别受影响的契约与验证范围，继续保留仍然成立的成果。持续演化要求这种局部更新能力，而不仅是完整流程的重复执行。

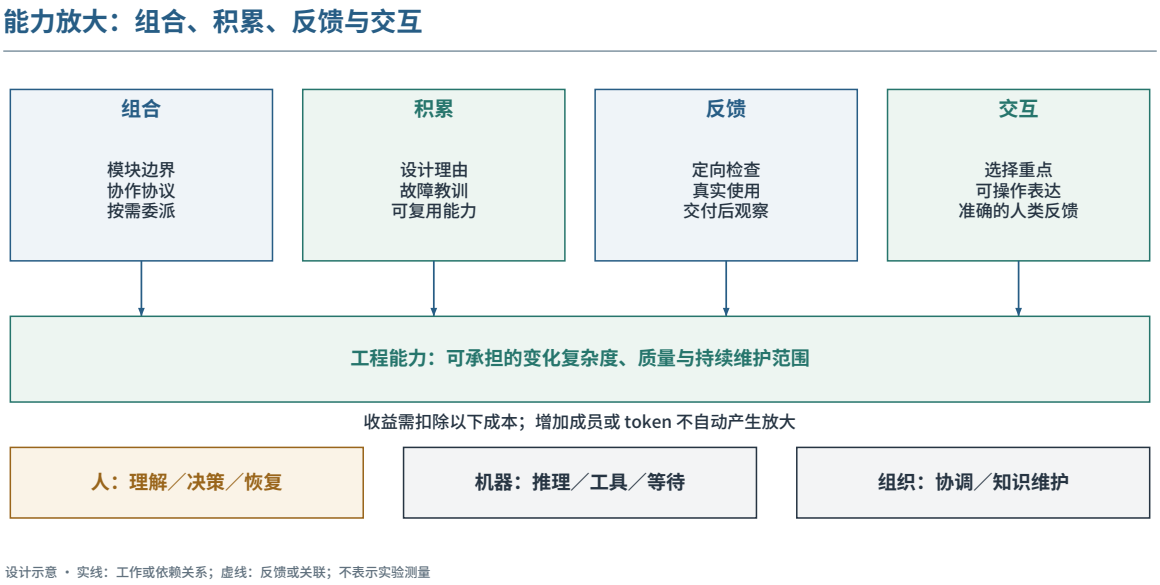


图 2 能力放大的四个来源与三类成本。各作用之间存在条件关系；收益需通过产物质量、变化适应能力和人的实际投入共同观察。图中关系是研究假设，不表示已测得的因果效应。

2.4 人的控制成本与系统资源

为便于讨论，可把人的参与成本表示为

$$C_H = T_{\text{理解}} + T_{\text{比较}} + T_{\text{表达}} + T_{\text{纠偏}} + T_{\text{恢复}}, \tag{1}$$

其中各项是相应活动占用的时间，恢复包括中断之后重建任务背景。这个分解用于提示观察项目，不假定不同人的每一分钟具有相同价值，也不要求所有项目都进行精密计时。

重要的是发现成本究竟发生在哪里：信息已经展示出来，仍可能需要大量恢复和解释工作。

系统侧成本包含模型推理、工具执行、等待、数据维护及协调。token 是其中一种可记录资源，并不直接等于金钱、能耗或认知工作量。不同模型的分词、缓存、输入输出价格和执行速度不同，比较时应同时报告模型配置、时间与费用口径。将人类时间和机器费用压成单个数字，需要明确权重及适用目的。

质量也需要多个维度。任务结果正确、用户体验顺畅、变化影响可控制、系统可维护，可能并不同步改善。本文主张在给定质量与资源约束下比较开发方式，必要时呈现多目标的可选结果。一个方式节省 token 却显著增加用户理解时间，不能仅凭模型成本更低就判断更优。

2.5 能力边界与收益发生的条件

可承担的复杂度包含多个方面：涉及多少独立职责、依赖之间是否存在并发与时间关系、需求变化能否局部处理，以及系统需要连续维护多久。一个文件很多但行为简单的项目，未必比文件较少却包含复杂状态协作的项目更难。比较能力边界时，应具体描述变化结构与质量要求。

组织增益可能随任务规模先上升后下降。少量独立工作可以从并行中受益，过多成员则可能增加协议协调和重复调查。长期经验也只有在新任务与既有职责存在联系时才容易复用。这意味着规模扩大并不自动带来更高放大率，研究需要观察收益的适用区间。

可以把一次开发组织选择理解为在质量约束下寻找合适的资源组合。某任务最需要减少人的理解时间，另一任务则受运行工具延迟限制。系统先识别主要限制，再调整成员、验证和表达投入。统一框架提供可选机制，具体选择由实际瓶颈决定。

2.6 组织自主性与可理解性

自主性意味着成员在目标和授权范围内选择行动、分工与检查方式。可理解性要求人在需要时能够获得足以判断的事实和解释。两者可以相互支持：清楚的目标与低成本反馈减少日常微观指挥，真实可查的结果使人能够接受更大的委派范围。

这里需要提供的是工作层面的解释，包括依据哪些约束、比较过哪些可执行方案、采取了什么操作以及观察到什么结果。系统无需把模型内部所有生成过程作为汇报材料。有效解释应能帮助人作出下一步判断，并与可核查的工作对象联系。

当模型能力增强时，一些原先需要多个成员协作的任务可能被一个成员承担。与此同时，项目也可能扩展到更长维护周期和更多依赖。框架的稳定部分是目标、证据、反馈和连续性；成员数量、上下文策略和表达形式则可以变化。持续放大能力意味着反复检验这种组合是否仍然带来工程收益。

3 需求与语义约束：在持续变化中保持软件方向

3.1 意图需要联系正在运行的软件

人的需求常以不完整的方式进入系统。“取消要可靠”“让我知道现在发生了什么”表达了关切，却没有确定所有状态与边界。智能开发系统需要把这些表达放回具体软件：谁在何时执行任务，哪些操作会重复，什么结果会让人相信任务已经停止。目标解释是逐步形成可行动理解的过程，其中既有可以依据已有信息解决的细节，也有必须由人决定的取舍。

有效澄清应围绕会改变行为的差异。例如，“取消”可能表示不再接收新步骤、尽快停止计算、撤销已产生的副作用，或仅关闭界面中的等待。这些含义对资源、数据和用户预期有不同影响。Agent 可以先检查已有契约和使用场景，再提出足够具体的选项。要求人一次写出完整规范，容易把工程分析工作重新推回给人。

意图还包括长期偏好。有的项目优先缩短故障恢复时间，有的项目对数据一致性有严格要求，同一项目不同功能也可能采用不同取舍。系统应保存偏好的范围和理由。例如，“局部修复通过定向验证即可交付”适用于已授权的维护范围，不能据此推出所有数据迁移都可以立即执行。明确范围让团队能够自主行动，并减少重复确认。

3.2 语义规范的内容与颗粒度

语义规范用人和 Agent 都能理解的语言说明目标、行为约束与成功含义。它可以包含实例、状态定义、接口契约、异常情境及重要的非功能要求。自然语言的优势是能解释动机和取舍；其含糊之处则需要通过具体情境和检查方式收敛。过于抽象的愿望难以指导实现，过于细碎的实现指令又容易随代码变化而失效。

对于取消功能，一份有用的规范可以说明：取消请求被接收后，用户立即看到处理中状态；只有在执行器确认停止或确定任务已经完成之后，才显示实际终态；重复请求应得到一致的解释；可保留的部分结果仍能访问。这些要求表达用户可依赖的行为，而具体由轮询、事件订阅或其他机制实现，可以由模块负责人决定。

表 3 从抽象需求到可讨论的语义约束

初始表达	需要明确的含义	可用于核对的情境
取消要可靠	请求接收与停止完成的区别	执行器延迟响应时界面如何变化
不要丢失结果	哪些中间产物可保留、何时可见	部分结果已写出时取消
操作要简单	用户需要几步完成并理解结果	不熟悉内部机制的用户首次使用
修复要及时	允许的交付范围与必要验证	旧版本异常时如何送达局部修复

规范的颗粒度可以沿职责组织。用户任务说明端到端成功含义，模块契约规定局部责任，稳定规则由代码检查反复核对。不同粒度的材料需要有明确关系，否则同一个词

可能在多个文件中逐渐产生不同含义。维护规范的目标是帮助下一次判断，而非使文档条目持续增长。

3.3 让目标、实现和证据可以互相定位

每次改进应能够回答三个问题：它服务于什么目标，改变了哪些对象，依据什么判断已经成立。对象可以是模块、接口、用户任务或界面状态；证据可以是代码测试、真实使用记录、运行观察或人类反馈。稳定的关联使成员能够按需深入，而不必重复扫描全部资料。

关联也需要反向可用。一个测试失败时，应能找到相关契约与责任模块；一个用户指出界面含义不清时，应能定位它显示的实际状态；某个协议改变时，应能识别哪些使用任务和投影受到影响。完整关联不要求为每行代码建立手工索引，优先维护那些会影响协作和判断的边界即可。

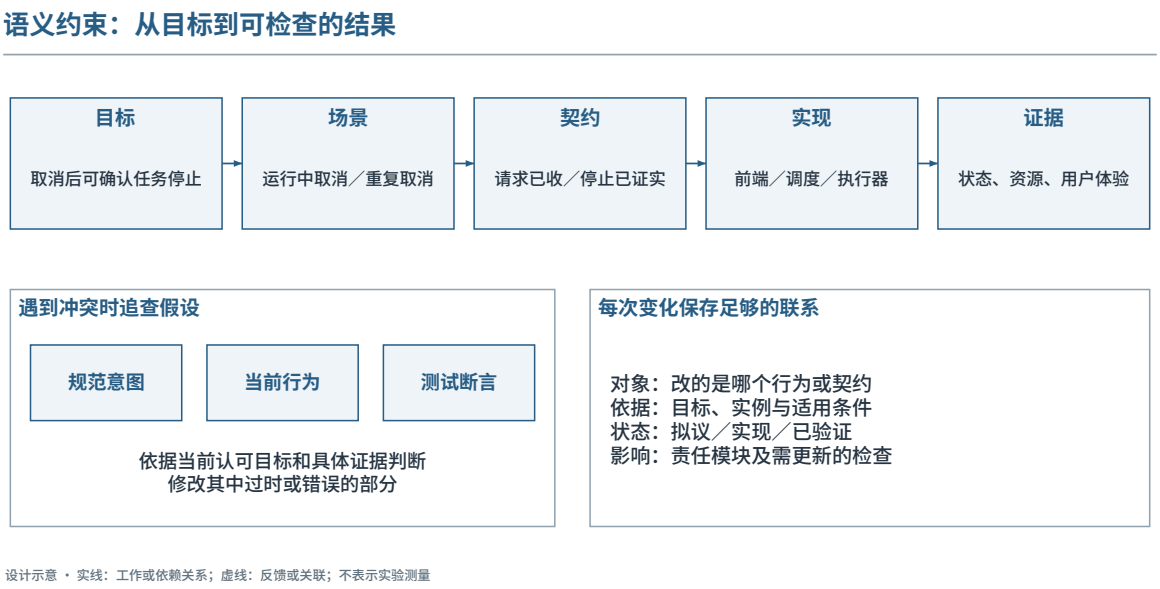


图 3 语义约束与工作证据的联系。需求经场景解释形成契约，契约进入实现与验证；出现冲突时，根据目标和证据定位需要修订的部分。

这套联系也为汇报生成提供事实基础。Agent 可以从目标出发选择已经交付的变化，从具体对象获取可展示实例，再关联尚未解决的问题。若汇报只依据成员口头声明，就可能把准备工作当成结果；若只依据代码差异，又可能忽略用户体验。需要将内容、运行和使用证据组合起来解释进展。

3.4 一个可使用的行为说明实例

为了让语义材料能够直接支持协作，可以围绕一个行为对象组织简短说明。对象名为“运行中任务的取消”，目标是让用户可靠表达停止意图并理解实际结果；参与方包

括用户界面、任务服务、执行器和结果存储；前置条件是任务已经创建且用户具备相应操作权限。行为说明随后列出接收、等待、停止确认及自然完成等情境。

这份说明还应包含可观察条件。请求接收可以通过持久记录核对，停止确认可以通过执行器状态与清理条件核对，用户理解则需要通过实际任务观察。不同证据支持不同结论，不能仅以接口返回成功证明任务已停止。明确这一点后，实现者与验证者可以独立选择合适工具。

当出现“任务取消后保留日志”的新需求时，说明只需更新结果保留部分及其关联情境。取消请求幂等性仍然是原有约束，未变化的部分继续有效。这种按对象与条件组织的材料有助于渐进维护，也为界面生成提供稳定语义。它不要求预先建立复杂的规范语言，最初可以用带明确字段和实例的文本实现。

3.5 规范、测试与实现发生冲突时

快速迭代中，规范可能落后于需求，测试可能保留过时断言，实现也可能偏离原意。发现不一致后，不能仅以“最新”判定正确。首先要确认各材料所描述的对象和适用条件是否相同，再检查哪项假设发生了变化。如果用户已改变输出保留要求，那么原先“取消后删除全部结果”的测试需要更新；如果用户意图未变，则当前实现可能存在缺陷。

修订应保留足以解释后续变化的理由。例如，把“已取消”拆成“取消请求已接收”与“停止已确认”，应说明它解决的是用户对实际执行状态的误解。理由可以关联一个复现情境和对应检查。这样，未来成员看到两个状态时，能够理解它们为何存在，也不容易为了界面简洁再次合并它们。

规范维护本身可以委派，但关键目标的变化仍需来自有效的人类输入或既定约束。Agent 可以提出新的解释和方案，并把影响具体展示出来；它不应通过不断调整成功定义，使所有实现都显得正确。评价系统时，也要防止执行者同时任意改写评价标准而掩盖失败。

3.6 工作状态与知识的不同生命周期

任务排队、运行日志、当前错误和一次测试结果属于工作记录；模块职责、稳定契约和经过核对的经验具有更长使用周期。将原始运行记录全部归档为长期知识，会增加检索噪声。相反，只保存简短结论而丢弃依据，也会让后来者无法判断结论是否仍适用。

一种可行做法是保留原始记录的入口，并提炼带条件的知识。经验条目可以包含“什么问题下有效”“采取什么处理”“怎样核对”“什么情况下需要重新检查”。同一经验在多个任务中被验证后，其适用范围可能扩大；遇到反例时则需要缩小或修订。知识的价值来自后续使用效果，而非被写入了多少文件。

WALL 类界面适合保存和关联设计点，但面向人的默认视图应根据当前问题组织信息。需要了解架构时，设计点附着在模块和协议上；需要验收功能时，它们附着在行为

与结果上。这样，完整信息仍可查询，人的日常参与则围绕有意义的情境展开。

3.7 把澄清作为持续协作的一部分

需求通常在看到具体成果后变得更清楚。因此，早期阶段应尽快形成能够被理解和体验的中间成果。它可以是协议示例、状态模拟或简化界面。人对这些对象的反馈，比孤立地回答抽象问题更容易包含准确约束。

持续澄清也意味着并非所有未知都需要立即解决。对当前实现无影响的远期问题可以明确保留；会决定模块边界或数据含义的问题则应优先处理。Agent 需要判断问题的重要性和时机，避免通过大量提问制造新的控制负担。其成功标准是理解逐步变准确，并能支持有质量的行动。

4 自主团队组织：架构分工、递归委派与长期经验

4.1 长期职责与临时任务

开发系统中的一些工作具有持续性，例如维护任务状态语义、执行器接口或用户交互。另一些工作具有临时性，例如完成一次跨模块改进、调查偶发故障或比较两个方案。组织时需要同时容纳这两种时间尺度。持续成员围绕稳定职责积累经验，临时成员则围绕当前任务聚集必要背景和协调关系。

长期沟通入口可以由 Leader 承担。它持续理解人的方向、偏好和项目承诺，并把工作委派给合适成员。Leader 也可以委派复杂任务的管理、验证跟踪和交付组织。若每次跨模块问题都必须回到顶层进行细节决策，组织会出现串行瓶颈，其上下文也会快速被执行信息占据。

因此，责任可以递归委派。任务负责人获得具体目标、约束和相关成员入口，再自行组织实现与验证。返回上层的信息聚焦成果、对原目标的影响和需要整体取舍的问题。细节继续保存并可按需访问。委派完成的依据是可核查的结果和明确的未完事项，而非成员数量或消息流量。

4.2 沿架构职责组织成员

Portal 的实践讨论倾向于沿功能、服务或包所体现的职责组织成员，以保持模块经验和接口连续性。与固定的需求、实现、测试阶段角色相比，这种方式可能减少同一背景在多个成员之间反复重建的成本。它是一项基于具体实践提出的组织假设，其适用性需要在不同项目中验证。

模块边界应依据职责与变化影响判断。一个目录包含若干文件，并不自动意味着它是合适的持续责任范围。可以观察某类变化通常由哪些知识决定、哪些接口会受到影响，以及是否经常迫使无关成员理解内部细节。若多个成员总要共同修改同一个内部决定，边界可能需要调整；若一个成员维护过多相互独立的领域，则拆分可能有益。

经典模块化研究将隐藏可能变化的设计决定作为分解依据之一 [1]。对 Agent 团队而言，职责边界还决定上下文应积累在哪里。执行器成员熟悉停止信号、资源释放和异常恢复；状态服务成员熟悉请求语义、幂等和终态判断。清楚的协议允许双方在局部深入，同时把跨边界的共同假设显式化。

团队图与软件架构图可以关联展示，但不必严格一一对应。一个成员可能维护多个小模块，一个复杂模块也可能需要临时协作。通信频率高也不一定意味着架构有问题：一次协议升级可能合理地需要密集沟通。判断应联系变化类型、重复发生的协调需求及维护成本。

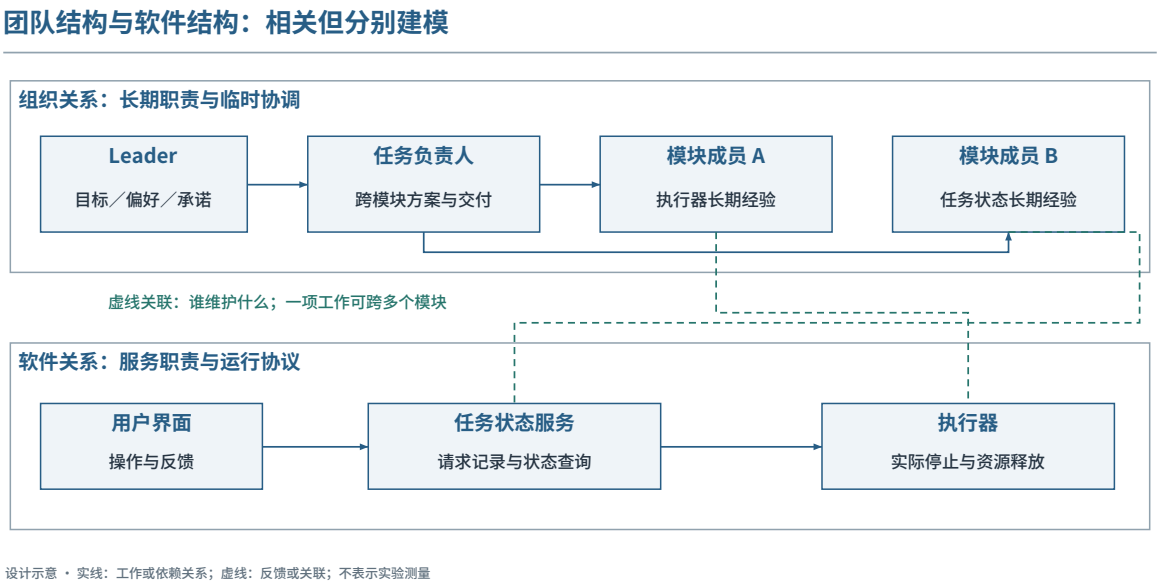


图 4 组织关系与软件关系分别描述。持续成员关联维护范围，临时负责人组织跨模块任务；组织图中的委派边与软件运行时依赖具有不同含义。

4.3 协议是自主协作的共同依据

模块之间需要明确可依赖的行为，包括输入、输出、状态、失败含义和兼容条件。协议应同时支持实现与解释。以取消为例，任务服务向执行器发送停止意图，执行器返回接收信息，随后报告实际终态。若各方对“接收”与“完成”的理解不同，单个模块测试都可能通过，组合起来却误导用户。

协议修改应联系具体调用方和使用任务。负责人可以先构造典型与异常交互，再让相关成员核对自身承担的部分。对于跨版本变化，还需要确定短暂共存期间的解释方式，例如旧执行器只提供一个状态时，界面如何避免过度承诺。接口稳定性并不要求协议永远不变，而是要求变化的条件和影响能够被理解与处理。

跨模块协调的结果应足够简洁，可被后续工作重新使用。记录共同决定、适用版本、重要例外和验证入口，往往比保存全部讨论更有用。若多个临时任务同时修改同一契约，持续负责人可以帮助统一解释，避免出现相互竞争的局部规则。

4.4 上下文分配与递归委派

上下文是稀缺的执行资源。它既占用推理成本，也影响成员注意什么信息。对复杂任务，合理的做法是把与局部判断有关的材料交给负责成员，把稳定结论、重要依赖和结果入口返回上层。所有成员接收所有日志，会使组织丧失分工带来的信息优势。

fork 可以作为派生任务上下文的一种手段，使新成员继承分出时的相关背景，再深入局部工作。但继承也可能携带过时假设和无关材料。派生时应明确目标、边界、所依据的状态以及需要返回什么结果。成员在分支中发现新事实后，不能假定父成员会自动

知道，需要通过合适的结果关联更新共享理解。

委派的收益可以粗略理解为节省的深入调查与并行等待，减去交接、协调和重新整合的成本。对于很小的修改，继续委派可能更慢；对于需要大量局部探索的任务，独立上下文更有价值。Leader 或任务负责人应能够改变任务粒度，并根据过去的实际成本调整组织。

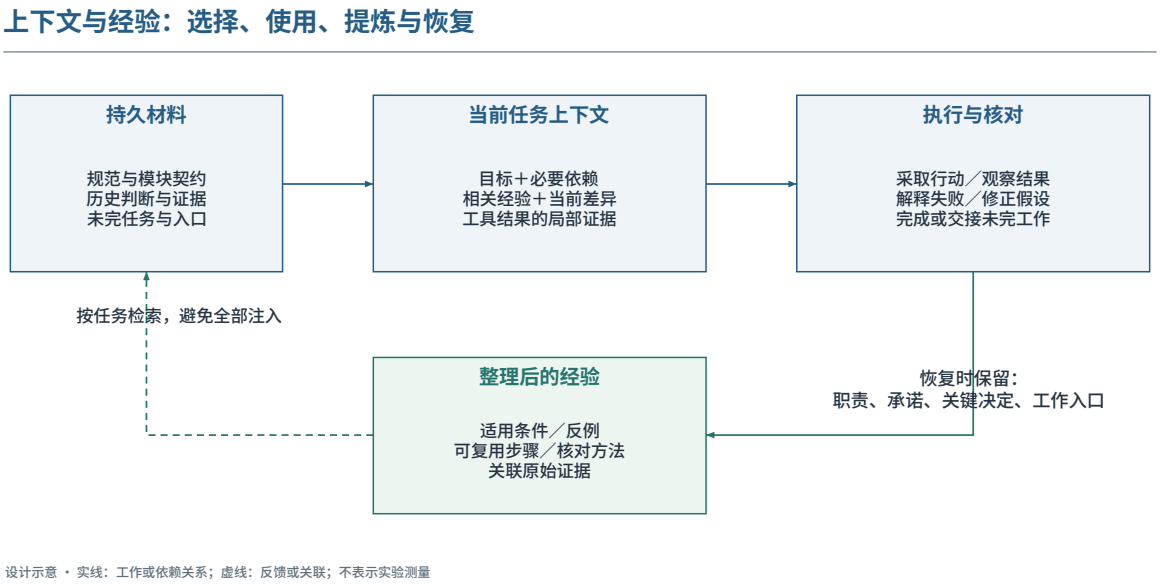


图 5 上下文与经验的生命周期。持久材料按任务进入当前上下文，执行结果经过核对后提炼为带适用条件的经验，供下一次任务和会话恢复使用。

4.5 连续身份、恢复与经验更新

持续成员的身份应与单次模型会话分离。模型更新、上下文压缩或执行中断之后，成员仍应能够找到自己的职责、当前承诺、重要决定和未完工作。恢复所需的信息应短而有结构，并保留深入材料的入口。只有一个“继续上次任务”的模糊摘要，往往不足以恢复实际状态。

经验可以分为对模块的稳定认识、具体问题的处理案例和可执行工具。稳定认识帮助快速建立背景，案例帮助识别相似问题，工具则将确定性操作固化为可重复能力。三者的维护方式不同。一个已失效的接口描述需要更新，一次罕见故障的历史证据仍可保留，一个脚本则需要随接口变化进行验证。

Reflexion 探索通过语言反馈和记忆影响后续尝试 [10]，Generative Agents 研究记忆、反思和规划在持续行为中的作用 [11]。这些研究提示外部经验可能改善后续行动。软件开发中的经验还需要与版本、契约和运行条件关联，其长期收益应通过后续维护任务检验。

成员退出时，成果与重要判断应能够被继任者使用。退出不应导致任务只有原成员才知道在哪里，也不应把所有会话历史无差别转交。可以以职责为单位保留当前契约、

已知问题、活跃任务和证据入口；继任者在真实任务中逐步补充必要细节。

4.6 并行工作中的协调边界

成员自主工作时，某些局部决定可能同时触及共享契约。协调应集中在这个共同边界，而不要求所有成员汇报全部内部过程。例如，界面成员准备合并两个状态，执行器成员则准备增加一种停止异常；双方需要先核对状态的用户含义，其余内部实现可以继续并行。

当两项任务竞争同一资源或改变同一承诺时，可以由临时负责人组织取舍。它需要了解相关目标、依赖和时间要求，而不必重新掌握全部模块实现。取舍完成之后，将共同结论返回责任成员，并保留影响范围。这个过程有助于避免顶层 Leader 成为每次局部冲突的中转站。

组织还应能识别工作停滞。成员可能已经完成局部实现，却在等待另一方确认协议；也可能因目标变化而继续执行已经失去价值的任务。共享状态应表达等待什么、由谁推进和何时需要重新判断。对人展示时可以进一步压缩为实际阻塞与可选行动，内部则保留足够的责任联系。

4.7 组织调整与独立核对

按模块组织不妨碍专业评审或独立验证。实现者通常掌握最多局部知识，但也可能沿用自己的错误假设。对跨模块行为、复杂协议或重要用户任务，可以让另一成员从实际入口核对结果。独立性应体现在信息来源与判断方式，而不只是角色名称不同。

组织本身也需要接受评价。重复调查、无效交接、等待顶层决定和共享契约冲突，都是可以观察的信号。系统可以据此合并职责、拆分过大范围或设置临时协调者。组织调整应解释要解决的具体问题，并观察之后是否减少成本，避免通过不断增加角色来处理角色之间产生的问题。

Conway 对组织沟通与系统设计关系的讨论 [2] 提醒我们，协作结构会影响软件边界。对于可动态创建成员的系统，这种关系提供了主动调整的机会。机会的实现仍依赖对实际变化的理解：组织图的整齐并不能替代软件内部职责的清晰。

5 持续开发与快速交付：在并行变化中推进演化

5.1 持续变化是工作情境

多个 Agent 同时工作时，共享代码和依赖可能一直在变化。若每次发布都等待所有工作完成，等待时间会随并行活动增多而上升。智能开发系统需要支持局部成果在具备交付条件时推进，并在之后继续吸收新的变化。持续集成强调频繁集成与反馈 [12]；本文进一步关注由 Agent 自主选择实际交付范围和完成修复的组织方式。

“局部”应依据行为与依赖定义。一个功能可能跨越前端、服务和配置，送达其中一部分会造成不完整行为。相反，同一文件中的多项相关改动有时难以低成本拆开，可以作为一个小批次共同检查与交付。责任成员需要判断本次实际内容能否形成可解释的行为结果。

在任务取消示例中，前端显示新的等待状态依赖后端提供相应语义。如果只发布前端，而后端仍把接收请求解释为完成，界面就无法可靠呈现结果。最小可用批次因此包含状态协议及其消费方；执行器内部与这项协议无关的优化可以留到后续交付。

5.2 确定性工具与智能判断的分工

范围判断、依赖理解和异常解释适合由 Agent 结合语义完成；实际内容捕获、并行合成、版本切换和结果记录应由可重复工具处理。把确定性操作交给稳定工具，有助于减少每个成员重新编排低层步骤的成本。工具应返回实际执行结果，使 Agent 能够继续判断，而不是只返回“已开始”。

一次交付可以包含准备、捕获、核对、合成、切换和观察。准备阶段允许不同任务并行，捕获阶段确定本次实际内容，最终合成与切换只在必要的短时间内协调。如果捕获之后又发生修改，后续修改进入下一批交付，不必无限等待代码停止变化。

并行合成需要保留他人已经交付的结果。对互不相关的变化可以直接组合，对同一契约或内容的冲突则应由相关责任成员解释。工具能识别文本冲突，却未必识别语义冲突；两个补丁都能应用，仍可能对状态含义作出互斥假设。定向检查需要覆盖这些实际共享关系。

5.3 验证与发布范围相互说明

交付记录应说明送达了哪些内容、依赖哪些前提，以及进行了什么检查。验证结果也应能定位到实际送达的内容和环境。若测试是在另一份代码上完成，团队需要判断差异是否相关。这个判断可以局部进行，而不必因任何无关变化就宣布全部证据失效。

定向验证的“充分”来自所要支持的结论。例如，一次修改只改变取消等待提示，可能需要检查显示和状态映射；若修改执行器停止过程，则需要检查资源释放与并发完成

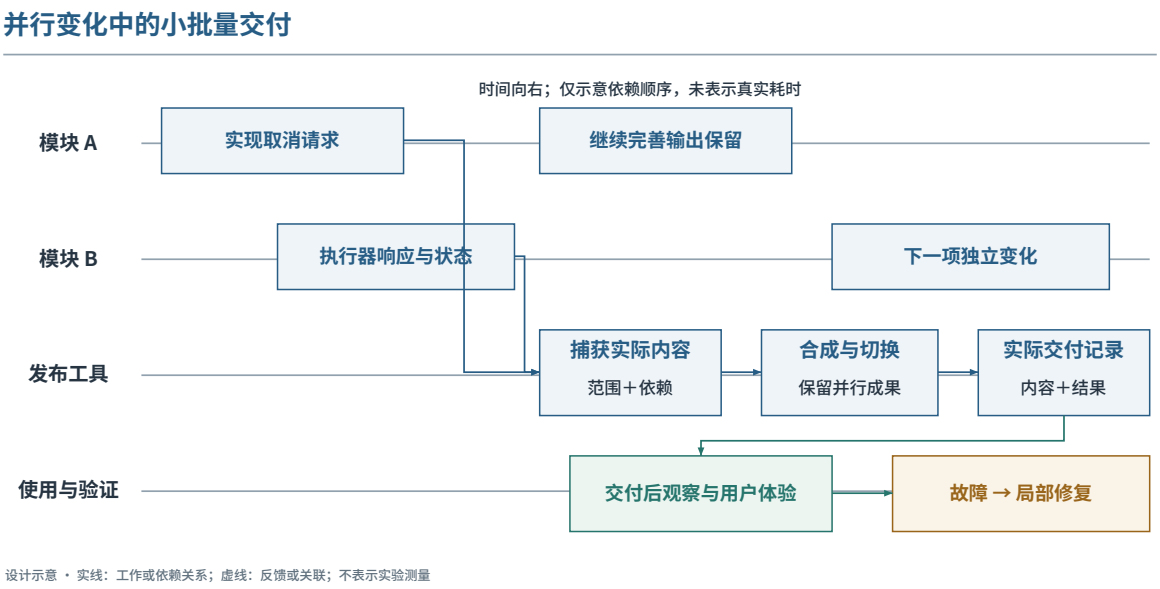


图 6 并行开发与小批量交付。各模块持续推进，发布工具捕获一组满足依赖的实际内容；后续变化进入下一批工作，交付后观察与修复继续进行。

分支。全量回归可以作为独立活动发现组合问题，触发条件由实际风险和成本决定。发布次数本身不能说明验证是否足够。

旧版本出现故障时，修复交付往往具有更高的时间价值。系统可以通过相同的局部捕获与核对路径送达修复，再观察是否恢复。修复本身也可能失败，因此需要保留实际结果和可恢复入口。快速交付的价值应体现在问题被发现到有效修复的时间，而非动作频率。

5.4 一次交付的最小事实记录

一次交付记录可以包含目的、实际内容标识、相关依赖、执行环境、验证入口和最终结果。它回答的是“什么变化实际到达了哪里”，而不只是“原计划发布什么”。如果工具只保存选定路径，却没有保存捕获时的内容，后续代码变化会使排障依据变得含糊。

例如，状态服务与界面共同交付之后，另一成员又修改了同一个界面文件。原记录应仍能找回先前送达的内容，同时把新修改识别为下一批候选。发生故障时，Agent 可以比较实际运行内容与证据，判断问题来自本批变化、共同依赖或后续修改。这样的记录支持局部核对，减少全局重建状态的成本。

交付记录也可以驱动面向人的进展视图。它提供已送达内容与证据入口，Agent 再根据目标说明用户可以体验什么。若切换失败，界面应显示失败及当前运行状态；若只完成准备，则不能把候选版本当成可用成果。工程记录与交互表达共享事实，有助于避免两个层面逐渐产生差异。

5.5 环境隔离与恢复的条件性取舍

开发入口和稳定入口可以共享部分数据或运行基础，也可以进行更强隔离。选择取决于系统架构、任务持续时间、数据约束和运维成本。共享有助于减少同步与迁移工作，却会使某些故障同时影响多个入口；隔离有助于限制影响，也会增加状态一致性和部署维护成本。

Portal 的实践讨论倾向于轻量滚动更新与快速修复，并接受部分运行基础共享。这项经验适用于理解设计取舍，不能直接推广为所有项目的发布规则。涉及不可逆数据操作或严格外部承诺的系统，需要相应恢复策略和更强检查。框架提供的共同原则是让约束明确、范围可判断、结果可观察。

代码恢复与数据恢复也需要分别考虑。重新启用旧代码不一定撤销新版本已经写入的数据。协议和数据格式的兼容条件应在变化前被识别，长时间运行的任务则需要明确在版本变化中怎样继续完成。保持任务身份和结果连续性，是判断交付是否成功的一部分。

5.6 低成本记录与交付后的学习

保留实际内容、交付目的、检查范围和运行结果，能够帮助后续排障与维护。这类记录可以由工具自动生成大部分内容，再由 Agent 补充必要解释。它们应服务于找回事实与恢复工作，避免把繁重的整理流程放到每次局部交付之前。

交付后的观察可能改变原先判断。一个修改通过代码测试，却使用户更难理解状态；另一个修改功能正确，但频繁引起跨模块同步。团队可以把这些结果反馈到验证策略、模块职责和界面设计中。持续交付因而也是获取工程知识的过程。

> **交付完成的含义**
>
> 一次交付至少要能够说明实际送达的内容、已核对的行为和仍未解决的问题。对于重要用户任务，还应提供可体验的结果。代码已经修改、工具已经启动或成员已经汇报完成，都只是其中的部分事实。

局部交付与长期架构质量应共同观察。若每次快速修复都增加特殊分支，后续变化成本可能持续上升。团队可以定期或在出现明显信号时整理重复问题，判断是否需要调整协议和模块职责。何时进行这类改进，同样可以由 Agent 结合实际收益选择。

6 自适应验证：语义规范、代码测试与真实使用

6.1 验证要支持什么判断

验证的价值在于减少对当前软件行为的不确定性。一次检查应能够说明它针对什么任务或契约、使用了什么内容与环境、观察到什么结果。对于持续变化的系统，验证结果具有范围和条件，不能把过去某次成功无条件延续为当前整个系统的可靠性。

代码式测试适合反复检查稳定且可精确表达的规则，例如请求幂等性、状态转移约束和权限边界。语义规范说明为什么需要这些规则，以及用户任务怎样才算成功。TAU 则让 Agent 从真实入口完成任务，检查多个模块组合之后的实际行为，并发现难以提前写成断言的体验问题。三者共同支持判断，维护方式也需要随变化调整。

真实任务基准为这一方向提供了参考。SWE-bench 将仓库与真实问题联系起来，要求系统修改代码解决问题 [13]；WebArena 与 OSWorld 分别构造网页和真实计算机环境中的任务评价 [14, 15]。这些工作表明，评价可以围绕环境中的任务结果组织。本文把这种任务视角用于软件开发中的持续验证，并关注验证发现如何进入修复和交付。

6.2 以完整用户任务组织 TAU

TAU 的选择单位可以是用户旅程。例如，创建一个长时间任务、观察进度、提出取消、确认资源释放并查看已保留结果，形成一条完整任务。它自然经过多个模块，也能暴露单元测试难以发现的状态解释问题。任务无需与每个设计点一一对应。

运行前需要明确初始条件、可使用入口、成功含义和可观察结果。若测试目标是普通用户体验，Agent 就不应依赖隐藏数据库修改来完成任务；必要的后台核对可以作为独立证据，同时说明它与用户可见体验的区别。测试数据应使关键分支可重复，例如让执行器延迟停止，观察界面是否过早显示终态。

Agent 的适应能力可能掩盖产品问题。它可能通过猜测按钮含义、刷新页面或绕过不清楚的入口最终完成任务。因此，TAU 记录应同时包含结果和过程中的摩擦：反复尝试、等待缺乏反馈、术语不一致、错误提示难以理解等。完成任务不能独自证明普通用户能够顺畅使用。

6.3 证据在变化后的有效范围

可将一条验证证据表示为

$$e = (s, v, c, o, a), \tag{2}$$

其中 s 为检查范围，v 为实际内容或版本，c 为环境条件，o 为观察结果，a 为结论依赖的假设。这个表示用于解释需要记录哪些事实，具体系统不必采用相同的数据结构。一次变化 Δ 是否使证据失效，要看它是否影响范围或假设，而不只看时间先后。

表 4 取消任务旅程中的不同验证责任

行为	低成本代码检查	真实使用需要观察的内容
重复点击取消	幂等键与状态不重复推进	用户是否理解请求已被接收
执行器延迟停止	非终态不被提前写为终态	等待反馈是否明确、是否误导
取消与完成并发	实际终态与版本一致	页面如何解释任务已经完成
保留中间结果	输出索引与访问规则	用户能否找到并理解保留内容

例如，取消提示文案变化通常不会使执行器资源释放测试失效；执行器停止逻辑变化则可能影响该结论，即使前端代码完全没变。依赖记录可以辅助选择，但不必假定它完备。当影响范围无法可靠限定时，扩大检查是合理选择。维护证据范围本身也有成本，需要与直接运行测试的成本比较。



图 7 局部证据复用示例。变化只使相关检查需要更新，其他范围的有效结论仍可保留；跨模块组合行为可以通过选定的真实任务补查。

在多人或多 Agent 持续开发中，测试执行期间仍可能出现新变化。结果可以说明它实际检查的内容，再对后续相关差异补查。让任何新修改都导致整轮测试作废，会形成难以结束的追赶。相反，把实际内容、依赖和未检查范围记录清楚，有助于逐步建立判断。

6.4 何时扩大验证

验证策略可以考虑变化重要性、已有覆盖、近期故障、组合未知和执行成本。单一局部变化可以只检查相关责任范围；多个模块连续改变同一用户任务时，值得检查组合路

径；基础协议变化且影响难以限定时，可以扩大到更广范围。上述情境是判断线索，不构成固定触发阈值。

全量代码测试和 TAU 的范围可以分别选择。某项目全量代码测试运行很快，精细分析用例反而更贵；而交互任务可能需要大量模型调用，适合优先选择核心旅程和变化最显著的路径。系统应比较实际成本，而不机械地把“更精准选择”视为更高效。

若以 $V(q)$ 表示检查 q 的预期判断价值、$C(q)$ 表示其资源成本，可以把优先级理解为在给定预算下选择有价值的检查组合。由于价值估计具有不确定性，实际系统应保留选择依据并根据结果更新经验。形式化表达帮助明确问题，不能替代工程证据。

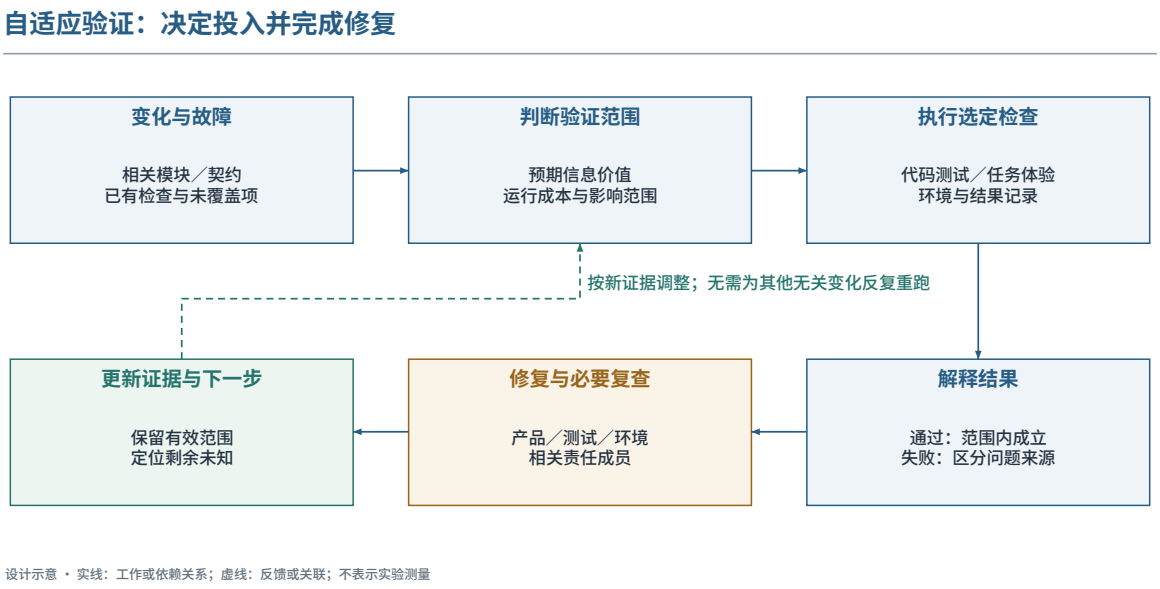


图 8 自适应验证的持续过程。范围判断、检查执行、结果解释、修复与证据更新相互连接；失败后的处理继续由相关责任成员完成。

6.5 三种不同变化下的投入选择

假设第一轮变化仅调整任务列表中的提示文案，已有状态契约和执行逻辑均未改变。可以核对文字含义、布局和状态映射，并复用执行器相关证据。若提示文案改变了用户对终态的理解，则需要增加一项聚焦的体验检查；“只改文字”本身并不能保证没有行为影响。

第二轮变化同时调整任务服务的重试策略和执行器响应方式。各模块局部检查通过后，仍存在组合条件未知，例如重试是否造成重复终态事件。这时以取消旅程组织一次相关 TAU，并核对事件幂等，可以更直接地获得判断依据。

第三轮变化替换了多个基础组件，依赖映射也不够完整。精细选测的依据变弱，可以运行较广的代码回归，同时选择几条核心用户任务。若全量代码检查成本很低，直接执行可能更合适。三种情境说明策略取决于需要消除的未知，而不是固定发布次数或固定日程。

6.6 从失败到修复完成

检查失败后，首先要区分产品缺陷、过时断言、环境问题以及测试操作本身的错误。区分依据可以包括复现、独立观察和当前规范。执行者不应为了得到通过结果任意放宽测试；也不应把所有失败都当成产品问题，造成无关修改。

明确原因之后，相关成员完成修复，重跑失败项和受影响范围，再通过既定交付路径送达。若问题反复出现且已经能够精确描述，可以增加低成本代码检查，减少以后重复支付探索成本。仍依赖界面感受和任务理解的问题，继续通过适合的真实使用方法核对。

修复跟踪需要停止条件。当前问题已得到解释、修复已核对、剩余未知被明确记录时，可以结束本轮工作。新出现的独立问题可以形成后续任务。无限扩展检查范围会消耗预算并延迟成果送达；过早结束则会让“发现问题”与“完成修复”脱节。

6.7 让验证任务与结果可以交接

一次 TAU 可以使用简短任务说明，把目标、初始条件、允许入口、成功含义和证据要求交给执行者。任务说明应描述用户需要完成的事情，避免预先写死每一步点击，从而允许发现新的有效路径和实际摩擦。对于必须核对的边界条件，则应明确给出相应情境。

表 5 可交接的验证任务与结果记录示例

字段	取消任务示例
目标与情境	创建长时间任务，在执行器延迟停止时取消，理解实际终态并查看保留结果
允许入口	用户界面完成操作；后台观察单独记录，不用于替代用户步骤
接受条件	请求已接收可见，停止前不显示停止完成，终态及结果可访问
观察材料	实际内容与环境、关键状态、操作序列、相关运行证据
结果解释	哪些条件成立，哪里需要猜测或绕路，失败涉及哪个对象
后续处理	产品、测试或环境问题的判断；修复责任与必要复查范围

结果记录可以同时服务于开发和汇报。模块成员需要复现条件与证据，人类读者可能只需要理解问题如何影响任务。二者共享同一事实入口，分别选择适合的表达。若结果仅保存为一段无法定位对象的总结，后续修复会再次支付背景重建成本。

任务和结果的交接也要保留未完成项。执行环境中断时，记录已经完成的步骤与当前状态，恢复后从可信位置继续；若初始条件已经变化，则重新判断哪些步骤需要重做。这样可以减少简单重跑，同时避免沿用已经不成立的结论。

6.8 验证的组织与可靠性

验证职责可以由 Leader 委派给专门成员，也可以由模块成员与独立体验者合作承担。重要的是检查方式能接触不同证据，并能把问题送回合适的责任范围。固定的角色名称无法保证独立性，多个成员共享同一个错误假设时仍可能共同遗漏问题。

评价验证策略时，应观察遗漏的重要缺陷、误报带来的返工、修复时间和每次检查获得的信息。只报告测试数量或通过率可能产生误导：大量简单检查可以全部通过，而核心用户任务仍然不可用。对未覆盖范围保持清楚认识，能帮助人和团队作出更合理的后续选择。

TAU 也需要接受人类使用的校正。真实用户可能采用 Agent 没有想到的路径，也可能无法理解 Agent 轻易推断出的术语。可以通过小规模用户任务观察检查这种差异，再调整验证任务与界面表达。由此，验证过程本身成为持续学习的对象。

7 智能交互与工作投影：让理解和控制共处

7.1 信息集中之后仍然存在理解成本

WALL 的实践说明，集中保存设计信息只解决了可查找性的一部分问题。面对几千个设计点，人需要判断哪些值得看；面对一句缺少背景的说明，人又需要重新建立语境。AI 在生成这句话时可能掌握实现、历史和依赖，而读者只看见结论。压缩文本减少了字数，却可能增加解释成本。

实际软件界面提供了另一种条件。人正在尝试某项任务，能够看见对象、操作和结果，因而可以直接指出哪里不符合预期。把这种能力扩展到开发过程，意味着为架构、协议、方案和进展提供同样具体的理解入口。视图的价值在于支持判断和行动，而不只在于展示更多数据。

因此，智能交互系统需要回答：当前人想理解什么，已经知道什么，还缺少哪些关键关系，以及理解后能够怎样参与。表达形式随问题改变。架构可能适合层次图，行为适合可操作演示，方案差异适合共同情境下的对比，进展则适合突出自上次查看后的重要变化。

7.2 工作语义与交互投影

工作语义包括目标、职责、模块、协议、任务、证据和结果等对象及其关系。交互投影根据人的当前问题选择其中一部分，组织成可理解的视图和操作。Lineage 可以承载团队与工作关系，Workbench 将相关内容投影为人的工作界面。软件架构、组织关系和运行状态仍有各自含义，需要通过明确关联组合。

投影过程可以分为信息选择、解释组织、媒介选择、操作绑定和结果更新。信息选择确定当前重点，解释组织提供背景与因果关系，媒介选择决定图、文字、演示或其他形式，操作绑定把人的动作关联到真实工作对象，结果更新则在工作变化后刷新相应表达。

同一个对象可以在多个视图中出现。任务取消协议可以在架构图中表现为模块之间的边，在机制视图中表现为事件序列，在用户旅程中表现为状态变化。对象身份和关键语义应保持一致，使人能够沿着疑问切换视图而不丢失背景。各视图不必显示相同细节，但应避免对实际状态给出相互矛盾的解释。

7.3 理解与控制发生在同一情境中

当人比较两个方案时，界面可以允许切换相同场景，查看行为差异，然后在该对象上表达偏好。发现某一步不符合预期时，可以标注那一步并说明期望。反馈自动携带场景、对象、当前状态和可见结果，减少人重新组织长段描述的负担。

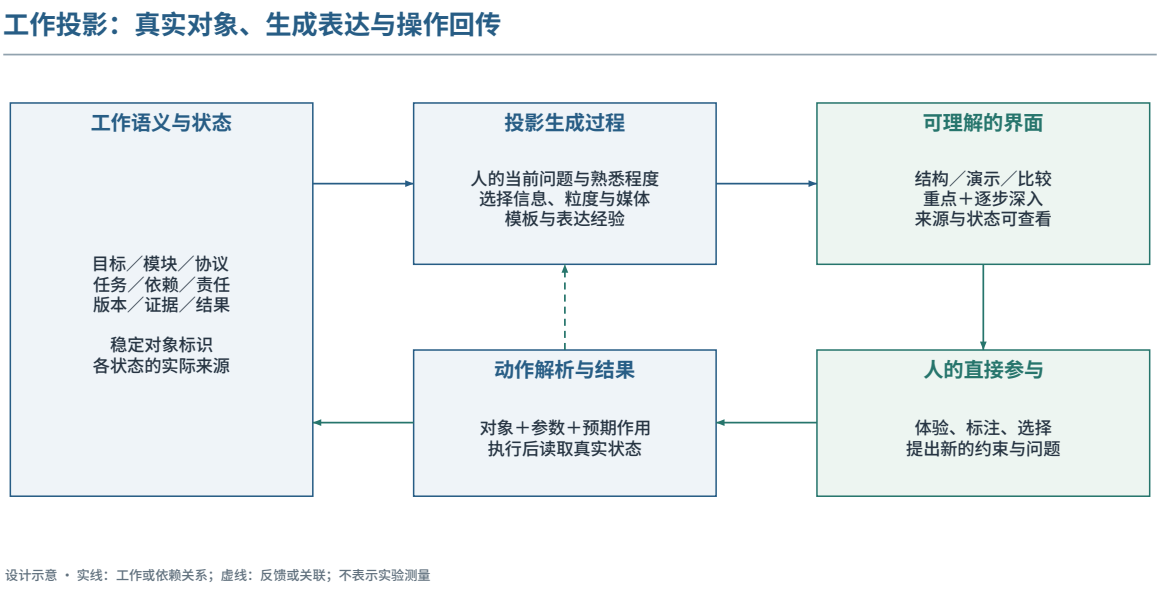


图 9 交互投影的生成与回传。视图来自工作对象和人的关注点，人的操作携带对象与参数进入实际执行，界面随后依据真实结果更新。

这种设计也能改善需求理解。例如，人体验取消演示后说“我希望看到停止确认之前的等待状态”，系统能够把反馈联系到当前状态映射，而不必猜测“那里”指什么。若反馈涉及软件行为，转入相应开发任务；若仅涉及解释形式，更新投影规则。两类工作可能相关，但需要知道实际改变了什么。

混合主动性交互研究讨论自动化服务与直接操作的结合 [8]。在本文的场景中，Agent 可以主动选择表达和提出建议，人则能够在需要时进入对象、调整约束和改变方向。自主性减少低价值的微观指挥，可操作表达帮助人保留关键控制。

7.4 表达必须说明状态与作用

生成界面可能看起来完整，却展示尚未实施的方案。因此需要让当前实现、拟议设计、模拟行为和已验证结果在相关位置清楚可辨。一个模拟取消按钮应说明它作用于演示环境；真实任务上的取消操作应关联实际任务，并依据真实执行状态更新。视觉相似不能代替语义准确。

状态说明也应避免过度承诺。请求已发送、服务已接收、执行器已停止和资源已释放，可能是不同事实。对用户有意义的界面可以简化细节，但保留下来的解释应足以支持正确判断。若某项结果尚未确认，可以呈现等待及可用的后续操作。

重要结论需要提供按需查看依据的入口。默认界面不必充满日志，但人应能在疑问出现时追查相关检查和实际对象。这种渐进式呈现与信息可视化中由全貌进入细节的思路相呼应 [7]。关键在于选择哪些细节有助于当前任务，而非简单折叠所有信息。

7.5 投影如何保持准确与连续

投影生成至少需要核对事实、语义和操作三个方面。事实核对检查当前状态是否来自有效记录；语义核对检查图示和说明是否保留了关键区别；操作核对检查按钮实际作用于哪个对象，以及结果怎样返回界面。三者中任一部分脱节，都可能让人形成错误判断。

例如，架构图可能仍显示旧的模块归属，但运行协议已经迁移。界面可以在相应对象上显示更新时间和需要刷新之处，并从实际对象获取新的关系。若人正在比较一个尚未实施的方案，则保留该方案的固定背景，明确它与当前运行状态的差异。持续更新并不要求把所有视图无条件切换成最新状态。

为了降低学习成本，可以保持对象名称、导航层次和基本动作的稳定性，将变化集中在当前问题的表达上。一个人多次进入任务服务后，应能找到相同职责与协议入口；新的演示则解释近期变化。这样，界面的适应性与长期方向感能够共同发展。

7.6 多媒体选择与生成成本

网页能够组合交互控件、状态视图和直接反馈；PPT 适合组织阶段成果与讲解顺序；视频适合展示时间变化和操作过程；3D 适合需要空间理解的对象。媒体选择应围绕理解任务，不因技术可用就增加展示复杂度。某个协议用几步可暂停动画就能解释清楚时，复杂三维场景未必带来收益。

多媒体还需要提供可访问的替代表达。动态图应允许暂停和回看，关键内容配有文字说明，颜色之外保留标签与结构差异。跨设备查看时，字号、布局和操作区域需要重新考虑。生成界面的质量包含这些使用条件，而不只是单张截图的美观。

生成成本可以通过复用稳定组件、局部更新和按需展开控制。工作事实变化时，受影响的视图需要更新，但不必重新制作全部展示。对很少被查看的细节，可以在用户进入时生成；对关键状态，则应保持及时可见。成本分配由使用频率、重要性和生成延迟共同影响。

7.7 异步汇报与表达的持续演化

人不会持续在线，汇报本身也需要时间。系统可以委派独立成员在阶段工作之后整理成果、更新界面和准备解释。人再次进入时，首先获得与上次查看相比的重要变化，再决定深入哪里。为了避免信息堆积，系统需要区分已经理解的内容、仍需回应的问题和新出现的影响。

界面可以通过反馈持续改进。人说“协议以后用具体交互来解释”，可以形成该类任务的表达偏好；人指出某个动态图过快，则调整播放与逐步观察方式。偏好应保存适用范围，允许后续纠正。不同读者和任务可能需要不同表达，个性化不应让核心对象失去共同含义。

人机 AI 交互指南强调能力说明、纠错与随时间变化的互动 [9]。本文据此提出一个工程化方向：表达生成作为持续工作，接收与软件开发相同的实际反馈。开发系统既改善软件，也改善人参与软件发展的方式。

投影的成功标准

人能够更快理解当前情况，在合适对象上表达准确反馈，并看见操作对实际工作的作用。界面的内容数量、动画数量和生成 token 数，都需要通过这些结果解释其价值。

8 UI 与汇报生成指南：从共同原则到典型表达

8.1 六项共同原则

第一，围绕人的理解和决策组织内容。生成前明确人需要知道什么、为什么需要知道，以及理解后可以做什么。首屏应建立足够背景并突出当前重点。对于架构讨论，重点可能是职责和依赖；对于验收，重点可能是可体验的变化和待判断的取舍。

第二，让理解和控制共处。视图中的对象应支持适合当前任务的操作，例如体验、比较、标注和选择。反馈携带必要上下文返回 AI。若某视图只用于讲解，也应提供继续提问或定位相关工作的入口，使解释能够支持后续行动。

第三，为信息提供背景和位置。设计点关联它涉及的模块、行为和目标，改进说明当前与拟议状态的差异。简短标签可以帮助导航，关键机制则需要实例和关系解释。对于熟悉背景的人可以压缩呈现，对首次接触者应允许展开基本概念。

第四，支持逐步探索并保持方向感。人可以从整体进入局部，从结果进入机制，从结论进入依据。深入后仍能知道当前位置和与整体的关系。跨视图跳转时保留当前对象、场景与筛选条件，减少重复定位。

第五，让表达与实际工作对应。当前实现、拟议方案、模拟结果和已验证事实具有不同状态，操作也需要明确作用范围。证据入口帮助人核对重要结论，状态变化则应来自实际执行结果。准确性是可理解性的基础。

第六，根据使用反馈改进表达。反复追问、错误理解、难以找到操作和中断后的恢复困难，都可以作为改进线索。常用概念与入口保持连贯，新的表达围绕具体问题展开。评价改进时观察理解与操作效果，而不单独追求新颖性。

8.2 指南一：层次化架构与协议探索

这类视图适用于理解系统组成、判断职责边界和评估改动影响。初次进入时显示主要功能或服务模块及其责任，避免同时展开全部内部细节。进入一个模块后，显示内部组成、关键数据和对外联系，并保留整体位置。

模块之间的边可以成为协议入口。点击之后，用一个请求如何经过双方、产生什么结果来解释契约，进一步展示异常和并发分支。协议字段可以按需展开，先让人理解双方各自保证什么。改进建议附着在受影响的节点和关系上，使变化具有可见的位置。

这类视图需要同时保留层次和跨模块关系。目录树适合定位内部组成，网络关系适合追查跨边界影响。两者可以通过按需展开结合。若人无法解释某个模块为何独立，或频繁需要穿越多个模块理解一个内部决定，视图可以帮助发现值得讨论的边界问题。

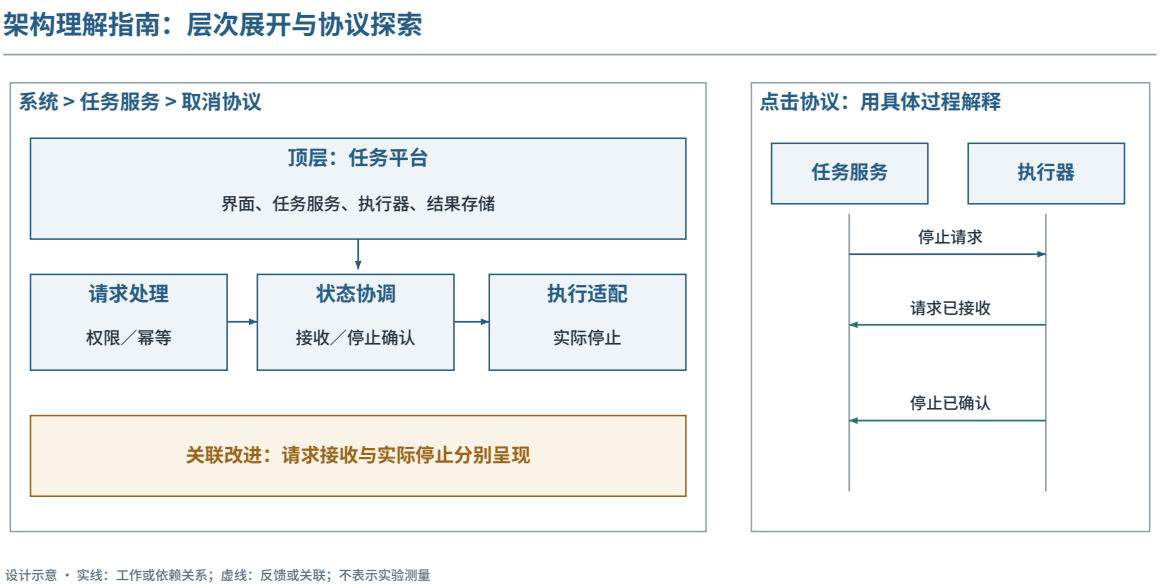


图 10 架构理解指南。左侧逐级呈现模块职责，右侧以具体交互解释选中的协议；拟议改进关联到它影响的对象。

8.3 指南二：围绕真实任务的功能体验

功能体验视图围绕一个明确任务组织操作、反馈和结果。例如，让读者启动一个模拟任务，提出取消，观察等待和最终状态，再查看保留输出。用户通过实际尝试建立理解，说明文字则解释关键差异和限制。

对比改进前后行为时，应使用相同任务条件。两个版本的不同输入会使人难以判断差异来自哪里。可以提供情境切换，例如执行器延迟、重复点击或任务已经完成。反馈入口附着在具体步骤上，使人能够指出“此时我以为已经停止”之类的体验问题。

评价这类视图时，观察读者能否独立完成任务、是否正确理解结果，以及反馈是否能够定位到可修改对象。演示如果过于理想化，可能隐藏真正困难的分支；因此需要让重要异常情境可以被尝试。

8.4 指南三：运行机制与可调情境

机制视图适用于解释状态、事件顺序、资源变化和失败处理。它可以采用时序图、状态机或逐步动画，并允许暂停、回退和调整条件。一个状态节点应说明进入条件和退出条件，事件边则说明由谁发出、携带什么含义。

机制解释的关键是将抽象规则放进具体过程。例如，取消请求与完成事件竞争时，改变二者到达顺序，观察终态如何确定。人由此理解系统为何需要区分接收与停止，而不必先阅读完整接口文档。模拟过程应明确其假设，实际运行证据则作为另一类入口。

人可以在异常分支上提出新的约束，例如资源释放失败时需要显示什么信息。这样的反馈直接作用于机制设计，并能进入相应测试任务。若动画只是沿箭头播放而无法解



图 11 可操作的功能体验示意。任务状态、演示操作、机制说明和针对步骤的反馈共处；图中的控件属于模拟演示设计。

释条件，它的视觉变化并没有增加多少理解价值。

8.5 指南四：共同场景下的方案比较

方案比较视图适用于需要人表达偏好或承担关键取舍的情境。候选方案应面对相同目标与场景，呈现行为、收益、代价和重要不确定性。对无法测量的效果应使用定性解释，并说明依赖条件，避免用精确数字制造确定感。

例如，取消后保留全部中间结果与只保留可解释部分，会影响存储成本、结果质量和用户预期。界面可以让人查看同一任务在两种方案下的结果组织，再选择偏好或补充约束。选择结果应关联其理由，便于后来在条件变化时重新评估。

比较视图不必要求人决定每个实现细节。对已被既定偏好覆盖的重复取舍，Agent 可以自行处理并在需要时解释；对影响目标或资源承诺的差异，再提供聚焦的选择界面。其价值在于提高判断质量并减少无关决策。

8.6 指南五：阶段进展与注意力分配

进展视图可以围绕人上次查看之后的重要变化展开，呈现可体验成果、仍然存在的阻塞和需要关注的取舍。完成状态应关联实际交付与验证范围，避免把“正在准备”“已经提交修改”和“用户可用”混为一谈。

人的关注点不同，进展选择也应不同。负责产品方向的人可能关心行为变化和使用反馈，关注架构的人可能关心职责与协议变化。系统可以提供默认重点和完整入口，让人按需展开。重点选择应能解释其理由，而不是隐藏所有不利结果。

操作可以包括体验成果、追查阻塞、调整优先级和回应问题。需要人回应的事项应足够具体，提供背景及可比较选项。大量“请确认”卡片会重新形成控制负担，因此日常工程判断应尽量在已授权范围内完成。

8.7 指南六：未来演化与依赖路径

未来演化视图在当前系统上叠加拟议变化，展示它们服务的目标、前置条件和可能影响。人可以切换不同路径，观察先做哪项工作会支持或限制后续改进。拟议节点应与当前实现具有清楚的视觉状态，避免让读者误以为能力已经存在。

这种视图可以把零散改进点组织成可讨论的路径。例如，先明确取消协议，再改善状态投影，随后增加可解释的输出保留策略。若某项基础工作能支持多个目标，其价值可以通过依赖关系解释；若某个方案需要大量前置修改，也应让人看见代价。

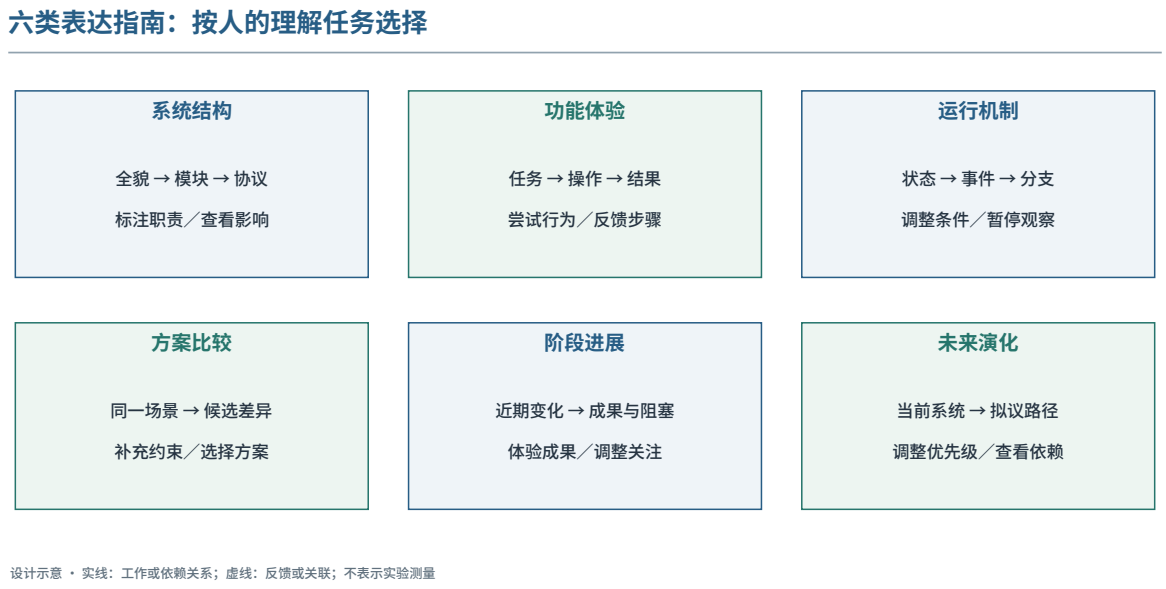


图 12 六类典型投影按理解任务组织。每类同时给出探索过程与人的参与方式；实际 Workbench 可以组合多个指南，并根据反馈改写。

8.8 把指南转化为可执行的生成任务

以架构指南为例，生成任务可以给出目标读者、当前疑问、相关对象、已知事实、可用动作和希望核对的理解。假设读者熟悉用户功能，却不了解执行器协议，任务重点就是让其理解“取消请求为何不等于已经停止”。所需输入是有关模块及协议、典型事件序列和一个延迟停止情境。

Agent 可以先生成模块层次与协议入口，再提供可暂停的事件演示，最后把反馈动作绑定到选中的状态。核对时请读者判断某一时刻任务是否已停止，并说明依据。如果读者只记住图上的颜色，却不能理解状态条件，就应修改解释和交互。任务以需要建立的理解约束结果，具体视觉实现仍保持开放。

同样的任务可以面向不同场景改写。阶段汇报可采用较紧凑的静态图与关键片段，深入讨论可提供可调参数的演示，移动设备上则可以按步骤展开。生成协议只需保持对象、状态与动作的一致含义，避免将某种布局固化为所有情境的唯一模板。

8.9 指南的组合、生成与修订

每份指南可以使用“适用疑问、首先呈现什么、怎样深入、人怎样参与、怎样评价”五项说明。它约束需要建立的理解，给具体布局和媒介组合留下空间。模板可以固化常用对象和交互结构，skill 可以说明生成与核对方法，协议可以规定状态与动作的语义联系。

生成时，Agent 先确定当前问题与事实依据，再选择适当指南，构造可查看的初版，检查关键文字和动作，最后根据人类反馈调整。若事实不足，可以生成标明假设的方案演示；若对象状态已经变化，则应更新相应部分。生成和核对本身也可以由不同成员协作完成。

跨视图组合应围绕人的疑问推进。例如，从阶段进展中的延迟问题进入机制演示，发现等待关系后查看架构协议，再比较两个修改方案。对象与场景随跳转保留，能够减少重复解释。有效形式无法穷举，指南应作为经验的积累方式，持续接受新的表达与反例。

9 综合案例：一次跨模块改进如何完成

9.1 案例条件与初始问题

本节构造一个以 Portal 类任务平台为背景的设计示例。场景中，用户提交长时间任务，可以查看进度、请求取消并访问结果。平台包含用户界面、任务状态服务、执行器和结果存储。该案例用于具体说明框架中的对象与过程，不表示一次真实生产故障的复盘，也不提供实验效果数字。

初始问题是：用户点击取消后，界面立即显示“已取消”，但执行器仍可能进行清理，资源也尚未释放。用户无法判断是否需要再次操作，更不清楚已有结果是否保留。虽然取消请求已经被成功接收，界面的表达却使用户形成了超出实际证据的理解。

人的初始需求可以表述为“取消后让我知道任务究竟停了没有，已有结果尽量保留”。Leader 将其联系到当前工作，识别出三个需要处理的部分：状态语义、实际停止和结果可见性。前两项是当前问题的核心，输出保留则需要明确何种中间产物能够被解释和继续使用。

9.2 形成语义约束与工作范围

团队先用正常、延迟、重复和并发四种情境澄清要求。正常情境下，用户请求取消并最终得到停止确认；延迟情境下，界面显示请求已接收且正在等待；重复情境下，多次请求不会造成相互矛盾的状态；并发情境下，如果任务已经自然完成，界面应显示实际完成结果并解释取消未改变终态。

输出保留的第一版约束可以是：已经正式写入并符合访问条件的结果继续可见，尚未完成的临时数据不作为完整结果展示。这个约束允许先解决状态误导，再由真实反馈决定是否扩展中间结果能力。未确定的部分被明确保留，而不会隐藏在“尽量”这样的模糊词中。

表 6 构造案例中的状态含义与可见行为

状态或事实	由何种事实支持	用户可以理解和采取的行动
运行中	执行器仍执行任务	查看进度或提交取消请求
取消请求已接收	请求已持久记录并进入处理	知道操作生效，继续等待确认
停止已确认	执行器报告停止，必要清理条件成立	查看终态和可保留结果
自然完成	完成事件已确定实际终态	查看结果，并理解取消发生得更晚
停止处理异常	实际停止尚未得到可靠确认	查看异常说明和可用处理入口

任务负责人据此组织一个跨模块改进。状态服务成员负责请求幂等与终态语义，执行器成员负责停止确认与资源观察，界面成员负责用户可见表达，验证成员核对完整任

务。长期职责保持清楚，临时负责人则跟踪跨边界协议与实际交付。

9.3 协议设计与并发条件

协议区分“请求已接收”与“停止已确认”。前者表示系统已经记录用户意图，后者表示执行结果满足终态条件。执行器延迟不应被界面误认为操作丢失，因此状态服务需要提供可持续查询或订阅的中间状态。重复请求应返回一致的当前事实。

取消与自然完成可能竞争。若完成事件先确立了终态，之后的取消请求不应覆盖真实结果。若取消先进入处理，任务仍可能在停止信号生效前完成。具体规则可以由状态版本、事件顺序和执行器确认共同支持，但共同目标是让终态解释与实际行为一致。

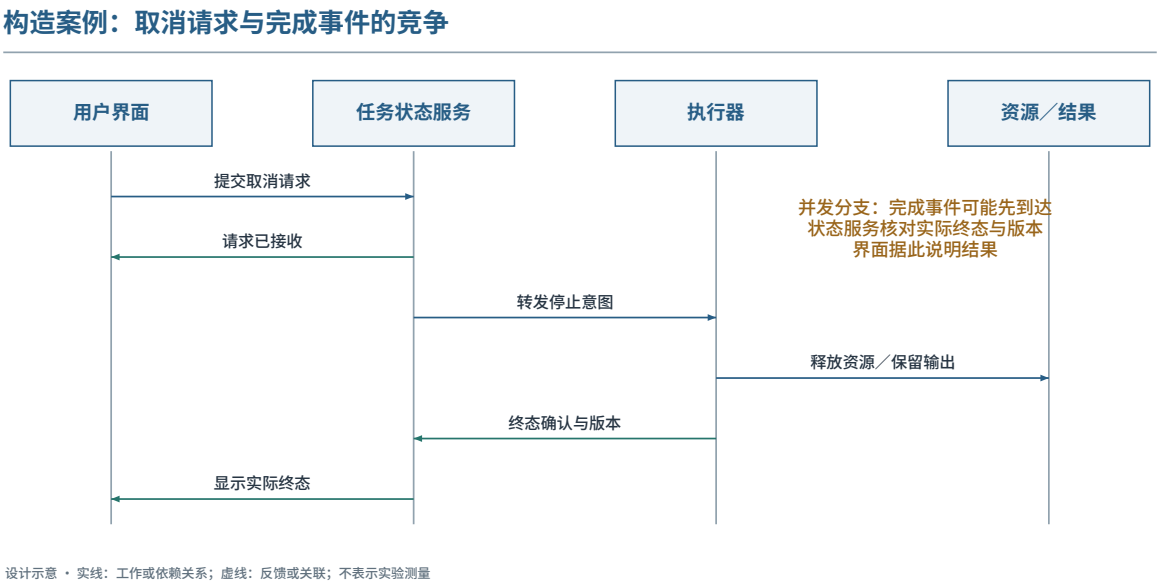


图 13 构造案例中的取消协议与并发分支。请求接收、执行器停止和界面终态分别由不同事实支持；完成事件竞争时，需要核对实际终态。

协议解释通过时序图进入 Workbench。人可以调整停止延迟，观察为什么出现等待状态；也可以让完成事件提前到达，查看终态说明。这种演示帮助人判断行为是否符合期望，并让模块成员以相同例子核对各自实现。

9.4 并行实现、检查与局部交付

状态服务与执行器可以并行修改，在协议边界上核对共同情境。界面则可以先使用模拟事件完成状态呈现，再连接实际服务。需要交付的批次包含状态语义和相应消费方；与此无关的页面整理或执行优化不必进入同一批次。

代码检查覆盖重复请求、终态不被错误覆盖以及停止确认的条件。TAU 从普通用户入口创建任务、提出取消、等待确认并访问结果。后台资源检查作为补充证据，核对实际停止与释放条件。两类证据分别说明内部规则与用户体验，彼此可以相互解释。

发布工具捕获实际内容并合成到当前运行版本，记录本次变化与验证范围。其他成员在此期间继续开发无关功能。若后续修改只影响任务列表的布局，已有停止逻辑证据仍可保留；若修改状态映射，则需要更新相关用户旅程检查。

9.5 一次失败怎样改变设计

在构造的首轮体验中，假设 Agent 能完成取消，但发现页面显示“取消中”之后没有说明等待什么，也没有提供结果入口。它通过刷新页面才看见终态。这意味着后端协议已经改善，用户仍然缺乏过程反馈。验证报告应记录这种摩擦，并关联到具体界面状态。

任务负责人将问题交回界面职责，补充等待说明和状态更新方式。若结果保留已经由服务支持，则把结果入口放在终态旁边。之后重跑相关体验和状态映射检查，无需重复检查所有无关业务功能。修复通过同一局部交付路径送达。

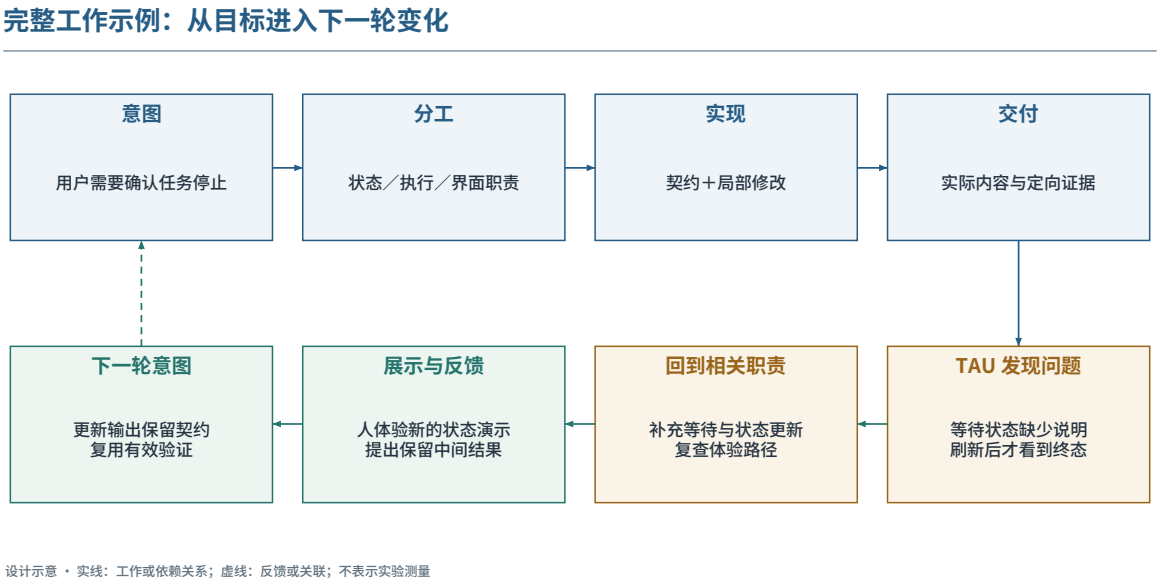


图 14 构造案例中的迭代过程。验证发现表达问题后，修复回到相关职责；人体验新结果并提出下一项需求，仍有效的工作和证据继续复用。

人体验新版 Workbench 后，可能提出“取消后希望看到已经产生的日志片段”。这是一项新的结果保留需求，而不只是界面提示调整。团队据此扩展结果契约，重新判断访问、完整性和存储条件。原有取消请求与终态验证仍可复用，新增检查集中在日志保留及其呈现。

9.6 案例留下的可复用资产

本轮工作可以形成几类资产：明确的取消协议、状态转移检查、可重复的用户任务情境、解释并发条件的演示，以及关于用户误解的经验。它们分别服务于实现、验证、沟通和后续学习。将这些材料关联到同一行为对象，有助于下一次修改快速建立背景。

同时，需要避免保存过度具体的临时知识。一次运行的全部日志可以保留为证据入口，长期经验则概括何种条件下会误导用户、如何区分状态以及怎样检查。若未来执行器架构改变，经验的用户目标可能仍成立，具体实现步骤则需要重新核对。

案例展示了框架的组合意义：语义约束帮助明确成功含义，架构职责支持并行深入，局部交付缩短反馈时间，TAU 暴露体验问题，Workbench 让人带着具体情境继续引导。每项机制仍有成本，组合后的净收益需要通过下一节的评价方法检验。

10 评价与资源分配：怎样判断系统放大了能力

10.1 从产出计数到工程结果

评价智能开发系统时，很容易获取代码行数、消息数、工具调用数和 token 数。这些量描述活动，但不能独自说明工程价值。更多代码可能增加维护负担，更多报告可能消耗人的注意力，更多测试也可能重复检查已经清楚的事实。评价应首先联系任务结果、使用质量和后续变化能力。

开发者生产率研究中的 SPACE 框架强调多维度观察，避免依赖单一活动指标 [16]。在 Agent 开发场景中，同样需要同时观察产物、过程、协作和人的体验。本文建议保留多个维度的结果，在明确质量要求和资源条件之后比较，而不是急于合成一个普适分数。

能力放大可以在受控条件下定义为相对于基线的增益。基线是直接把相同目标交给一个具备相同工具和资源条件的强 Agent。组织方式增加之后，若交付质量提高、人类时间下降或可承担复杂度扩大，则可以进一步分析收益来源。基线也应经过合理配置，避免用弱基线制造优势。

10.2 核心评价维度与记录口径

表 7 建议的评价维度与观察对象

维度	主要观察对象	解释时需要注意
任务结果	接受条件、实际行为、未完成项	保持独立检查，防止成功定义漂移
使用质量	完成任务、错误理解、绕路和等待	Agent 成功不等于普通用户顺畅
演化能力	后续修改成本、影响范围、返工	需要多轮任务与维护观察
交付效率	需求到可用、故障到有效修复的时间	区分执行、等待与恢复时间
人类负担	理解、比较、表达、纠偏、恢复时间	同时观察理解正确性与决策质量
系统投入	token、费用、工具时间和协调开销	记录模型、缓存与计费口径

对人类负担的观察可以使用任务计时、操作记录与简短理解题。比如，阅读汇报后请参与者判断取消是否已经完成、指出某项协议的责任方，或选择符合给定目标的方案。若阅读更快但理解错误更多，不能视为改进。主观满意度有价值，但需要与实际任务表现一起解释。

架构质量可以通过后续变化体现。给系统加入一项相关但未提前公开的需求，观察需要理解和修改哪些模块、是否破坏既有契约，以及是否增加特殊分支。代码结构指标可以辅助描述，真正重要的是局部变化是否迫使无关范围同步调整。

10.3 多轮任务与比较条件

一次功能实现不足以评价长期组织。可以设计连续任务序列：新增能力、改变约束、处理跨模块故障、完成交接，再实现另一项依赖此前经验的需求。各条件从相同初始状态开始，使用相同任务和外部评价。后续任务需要避免被某条件提前获得，以便观察真实适应能力。

比较条件可以逐步增加机制：单 Agent 基线；沿模块组织的团队；团队加持续经验；团队加智能交互；完整框架。消融实验可以去除某个机制，检查观察到的收益是否仍存在。不同条件应记录实际资源，并同时报告预算固定和质量目标固定的比较方式。

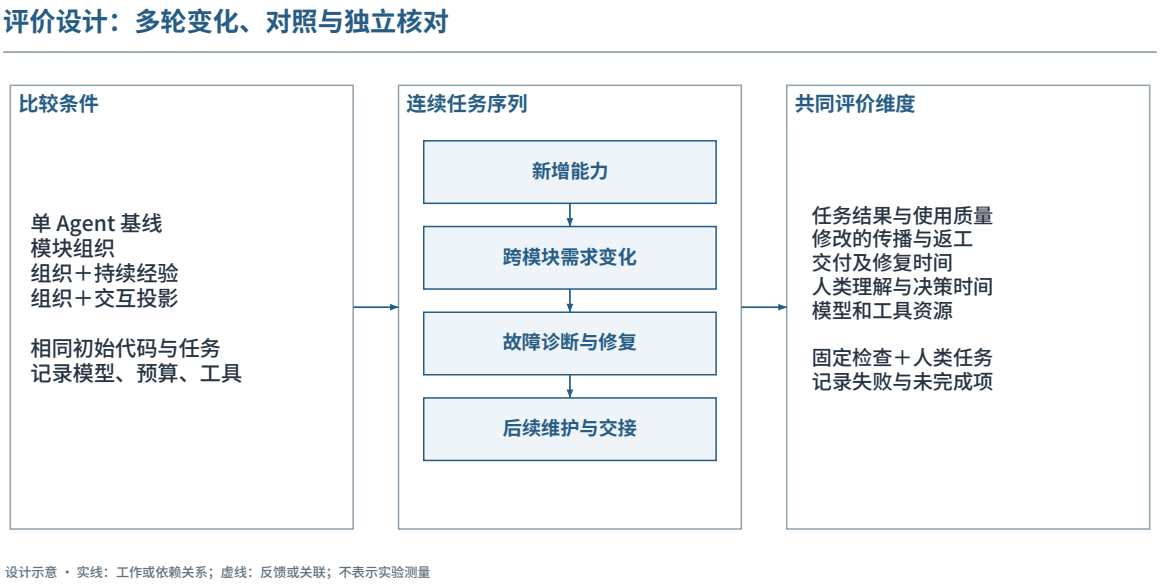


图 15 多轮演化评价设计。不同组织条件面对相同任务序列，以独立检查和人类任务共同评价结果，并记录资源与失败。

人类参与实验需要考虑学习效应。参与者看过一个界面后，会对同一问题更熟悉，因此不同表达方式的顺序应平衡，或使用难度相近的不同任务。开发系统也可能因随机生成和工具波动产生差异，需要重复运行并报告分布，而不只挑选最佳案例。

模型升级后的评价应同时保留可比任务和增加更复杂任务。前者帮助了解同一工作是否变得更高效，后者帮助观察方法是否扩展能力边界。若只保留已经变得容易的任务，可能低估组织方法的潜在作用；若只增加难度，又难以区分模型与任务变化带来的影响。

10.4 展示与交互可能成为主要投入

本文提出一个值得观察的成本结构假设：面向人类理解和控制的 token 消耗，可能与代码生成处于相当规模。这类工作包含选择重要信息、解释背景、生成交互演示、核对状态和根据反馈更新表达。它们可能贯穿整个维护周期，尤其在软件复杂度扩大之后持续发生。

可以把模型投入按活动记为

$$T = T_{\text{实现}} + T_{\text{验证}} + T_{\text{协调}} + T_{\text{表达}} + T_{\text{其他}}, \tag{3}$$

并观察 $T_{\text{表达}}/T_{\text{实现}}$ 随任务、用户熟悉程度和复用策略怎样变化。这个比例是描述量，不是预算目标。同一次调用若同时服务多个活动，应采用一致归类规则，或明确记录共享部分，避免重复计数。

表达投入的收益可能通过人的理解时间下降、反馈更准确和后续返工减少体现。一个生成演示需要更多 token，却帮助人提前发现关键误解，可能降低总成本。反过来，频繁生成无人查看的复杂报告，即使内容正确也可能没有足够收益。评价需要把使用和后续结果联系起来。

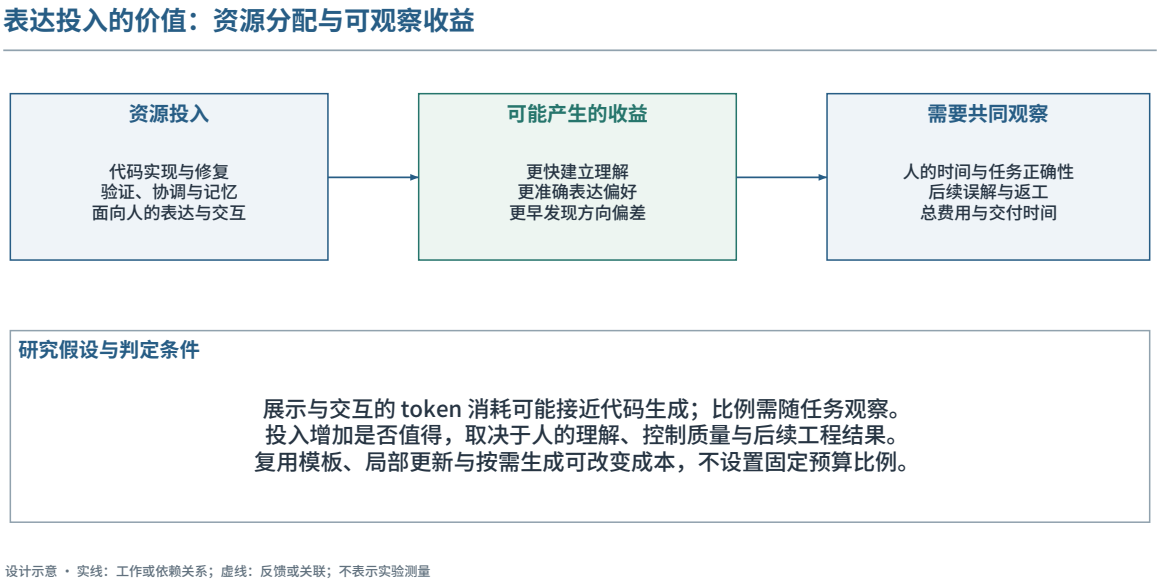


图 16 表达投入的价值假设。增加生成资源可能改善理解与反馈，但是否值得需要通过人的任务表现和后续工程结果判断；图中没有预设最佳比例。

成本还受复用方式影响。稳定模板、原生组件和已验证示例可以降低后续生成成本；局部更新可以避免重复制作未变化内容；按需生成适合少用的深入视图。不同策略可能同时改变等待时间与用户体验，因此应把延迟和资源一起报告。

10.5 可检验的研究假设

第一项假设是，沿稳定职责组织持续成员能够减少后续任务的背景重建与跨阶段交接。可以观察调查时间、重复工具读取、契约冲突和维护质量。若模块边界不清，或者任务变化完全跨越既有职责，这种收益可能减弱。

第二项假设是，基于变化与证据选择验证范围，能够在保持重要缺陷发现能力的同时减少无效检查。需要同时记录漏检、误报、检查费用和修复延迟，避免只比较运行次数。对于全量测试极便宜的项目，自适应选择的管理成本可能高于收益。

第三项假设是，把理解与控制放在同一界面，能够提高反馈的准确性并降低上下文重述成本。可以比较纯文字汇报、固定仪表盘和按任务生成的交互视图，让参与者完成相同理解与决策任务。若生成视图经常改变结构，学习成本可能抵消优势。

第四项假设是，持续经验与智能交互的收益会在多轮维护中累积。验证它需要较长观察，并控制模型、工具和任务顺序的变化。一次成功展示只能说明该情境可行，不能证明长期收益已经成立。

10.6 一个可实施的首轮研究方案

首轮研究可以聚焦交互投影与多轮维护的组合。选择具有清楚接受条件的任务平台，准备若干难度相近的变化序列，每个序列包含实现、需求调整和故障修复。先在固定模型与工具条件下运行单 Agent 和模块团队，再在相同成果上比较文字汇报与交互投影。这样可以分别观察组织和表达的作用。

参与者接收明确目标，例如确认某项行为能否使用、找出一个故意保留的解释缺口，以及提出符合目标的下一步改进。记录完成时间、判断正确性、反馈能否被执行者准确定位，以及执行者据此修改所需的额外澄清。界面的生成时间和费用也纳入记录，才能计算整个协作过程的成本。

研究应在开始前明确主要结果和停止条件，并报告全部完成与未完成运行。样本量可依据预实验中的波动和希望识别的效应确定，本文不预设任意人数。分析时同时展示差异分布和典型失败过程，使读者能够判断收益来自哪些条件，而不只看到一个平均分数。

10.7 哪些观察会要求修改框架

一个可检验框架需要说明什么结果会削弱其主张。若模块团队在多轮任务中持续花费更多协调时间，且没有改善结果或维护质量，就应重新判断职责划分是否有价值。若长期记忆增加了错误假设的复用，则需要调整经验维护，而不能仅以“积累不足”解释所有负面结果。

同样，若一个更强的单 Agent 在相同条件下表现更好，框架可以采用更简化的组织，同时继续使用目标、证据和交互机制。能力放大的目标要求机制接受这种调整。研究应报告失败和适用边界，使下一次设计能够选择更合适的组合。

10.8 效度威胁与结果解释

任务选择、评价泄漏、模型随机性和参与者经验都可能影响结论。开发者提供的示例也可能更适合某种组织。评价应公开任务条件与失败情况，保留独立接受条件，并记录因环境问题未完成的运行。环境失败与产品失败应分别描述，但不能简单删除所有不

表 8 负面结果及其可能要求的设计调整

观察	需要进一步区分的原因	可能的调整
团队协调超过并行收益	边界不清或任务过小	合并职责、减少层级、改变委派粒度
生成界面阅读更快但判断更差	信息缺失或视觉暗示误导	补充关键条件，重新组织状态解释
自适应选测仍遗漏重要缺陷	依赖信息不完整或选择偏差	扩大检查，修订证据关联
经验复用导致重复错误	条件过时或缺少反例	缩小经验范围，增加核对入口
展示投入增长却很少被使用	关注点不符或生成过度	按需生成，减少重复更新

利样本。

对结果的解释需要关注适用范围。一个小型项目的组织开销可能很高，一个长期多模块系统则可能受益明显。一个熟悉系统的用户可能偏好紧凑图示，首次接触者需要更多实例。报告这些条件，比宣称某种流程普遍优越更有帮助。

本文提出的评价方案尚未在本稿中执行。后续研究可以先用少量完整任务验证记录与测量是否可行，再扩大项目和参与者范围。评价过程本身应避免成为新的高成本仪式，优先选择能区分主要假设的观察。

11 开放问题与未来方向

11.1 长期方向与组织适应

长期智能开发仍面临目标漂移问题。人可能改变偏好，团队可能把局部指标当成最终目标，历史经验也可能固化已经失效的假设。系统需要能识别重要分歧，并在合适情境中让人重新判断。交互投影可以降低这一过程的成本，但其选择机制本身也需要可检查，避免只呈现容易解释或已经成功的部分。

组织适应同样是开放问题。成员能力增强后，合理职责范围可能扩大；项目复杂度增加后，又可能需要更多专业协作。如何根据真实工作调整团队，同时保持知识连续性和接口稳定，需要长期观察。临时成员与持续成员之间的边界，也可能随任务和模型条件变化。

经验维护需要处理可信度与遗忘。系统应保留有用的条件化知识，能够在反例出现时修订，并降低过时材料进入当前上下文的概率。记忆容量增加不能直接解决这一问题，关键仍是选择、核对和使用过程。未来可以研究经验在不同模型和团队之间迁移时的适用条件。

11.2 生成交互的稳定性与个体差异

生成界面的有效形式难以穷举。高层原则与典型指南提供起点，具体效果需要在人类使用中形成。持续变化的界面可能改善特定任务，却增加定位和学习成本。如何保持稳定的对象语义与常用入口，同时允许局部表达变化，是值得深入研究的设计问题。

不同人的背景和参与方式也会影响表达。架构负责人、产品使用者和临时审阅者可能关注不同问题。系统可以根据当前任务和明确偏好调整视图，但需要避免将过去一次选择当成永久规则。表达经验应能够被人纠正，并在共享协作中保持可交流的基础。

11.3 运行模块与开发成员的进一步结合

更远的方向是，软件运行模块本身具有智能能力，并参与自身或相关模块的开发。一个运行中的任务管理 Agent 可能持续观察使用问题，提出协议改进，并与开发成员协作。这会使运行反馈与开发知识更加接近，也会增加职责与变更边界的复杂性。

研究这种结合时，需要分别理解当前运行承诺和未来修改提案。运行模块提出的建议可以进入开发与验证过程，实际变化仍需依据明确目标和证据实施。自我修改是否带来收益，要看它能否改善真实任务、保持可理解的状态，并避免局部优化损害整体目标。本文将其作为远期研究方向。

11.4 结语

智能软件开发的长期目标，是让不断增强的 AI 能力通过组织、积累、反馈和交互转化为更大的工程能力。软件结构决定局部工作能否有效组合，持续成员让经验跨任务延续，局部交付和自适应验证使判断接触现实，而智能交互帮助人以较少投入理解和控制系统的发展。

这套框架中的具体机制可以变化。更强模型可能减少某些交接，更好的工具可能降低验证成本，新的表达方式可能改善人的参与。持续有效的评价问题是：系统是否更准确地理解目标，是否更自主地完成有质量的工作，以及人是否更容易发现偏差并作出有效引导。围绕这些问题开展实现与经验研究，可以逐步形成随 AI 能力共同发展的软件工程方法。

参考文献

[1] David L. Parnas. On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15(12):1053–1058, 1972. https://doi.org/10.1145/361598.361623.

[2] Melvin E. Conway. How Do Committees Invent? *Datamation*, 14(4):28–31, 1968. Author-hosted text: https://www.melconway.com/Home/Committees_Paper.html.

[3] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*, 2023. https://arxiv.org/abs/2210.03629.

[4] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024. https://arxiv.org/abs/2405.15793.

[5] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, et al. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. 2023. https://arxiv.org/abs/2308.00352.

[6] Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. 2023. https://arxiv.org/abs/2308.08155.

[7] Ben Shneiderman. The Eyes Have It: A Task by Data Type Taxonomy for Information Visualizations. *IEEE Symposium on Visual Languages*, pp. 336–343, 1996. https://doi.org/10.1109/VL.1996.545307. Author manuscript: https://www.cs.umd.edu/users/ben/papers/Shneiderman1996eyes.pdf.

[8] Eric Horvitz. Principles of Mixed-Initiative User Interfaces. *CHI*, pp. 159–166, 1999. https://www.microsoft.com/en-us/research/publication/principles-mixed-initiative-user-interfaces/.

[9] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, et al. Guidelines for Human-AI Interaction. *CHI*, 2019. https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/.

[10] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. 2023. https://arxiv.org/abs/2303.11366.

[11] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. *UIST*, 2023. https://arxiv.org/abs/2304.03442.

[12] Martin Fowler. Continuous Integration. 2006; revised 2024. https://martinfowler.com/articles/continuousIntegration.html. Accessed 2026-10-15.

[13] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR*, 2024. https://arxiv.org/abs/2310.06770.

[14] Shuyan Zhou, Frank F. Xu, Hao Zhu, et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. 2023. https://arxiv.org/abs/2307.13854.

[15] Tianbao Xie, Danyang Zhang, Jixuan Chen, et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. 2024. https://arxiv.org/abs/2404.07972.

[16] Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler. The SPACE of Developer Productivity: There's More to It than You Think. *ACM Queue*, 19(1):20–48, 2021. https://doi.org/10.1145/3454122.3454124.